



Advanced Platform-Level Interrupt Controller (TT-APLIC) Specification

Version ASCIIDOC Conversion, 06/2025: Pre-release version

Table of Contents

1. Overview	1
1.1. Implementation Notes	1
1.2. Repositories	1
1.3. TT-APLIC Functionality	2
1.4. Versions of Source Code and Tool	2
1.5. Verification Result Summary	2
2. Interface Specifications	3
2.1. Parameters	3
2.2. Input/Output Ports	3
3. Architecture	5
3.1. Internal TT-APLIC Structure	5
3.1.1. Control Region	6
3.1.2. Request Handler	7
3.1.3. Gateway	8
3.1.4. Register Control	9
3.1.5. Target Notifier	10
3.2. Function	10
4. Timing Chart	11
5. Verification	12
5.1. Verification Environment	12
5.2. Common Smoke & Random Test Initialization Process	12
5.2.1. Initialization Procedure	12
5.2.2. Initial domaincfg Values for Testing	13
5.2.3. Initial sourcecfg Values for Testing	13
5.2.4. Initial target Values for Testing	13
5.2.5. Initial setie Values for Testing	13
5.3. Smoke Test	14
5.3.1. test 1: Updating and Reading configuration registers	14
5.3.2. test 2-5: Forwarding MSI with Level/Edge-Sensitive and Active High/Low Interrupts	15
5.3.3. test 6,7: Handling Multiple Interrupts in the one/multiple domain(s)	15
5.3.4. test 8,9: Handling Simultaneous Interrupts and Config Requests in M/S-domain	15
5.3.5. test 10,11: Handling genmsi in M/S-domain	16
5.3.6. test 13-18: Forwarding msi Using setip(num(_le)) in M/S-domain	16
5.3.7. test 19: Stopping MSI Forwarding in disabled domain	16
5.4. Random Test	18
5.4.1. Generating random data for read/write requests	18
5.4.2. Generating random addresses for read/write requests	18
5.4.3. Generating random data for wstrb	19
5.5. Coverage Test	20
5.5.1. Results of Line, Branch, Condition, and Toggle Coverage.	20
5.5.2. Waiver for Line, Branch, Condition, and Toggle Coverage.	20
Line Coverage	20
Condition Coverage	21
Toggle coverage	21
5.5.3. Functional Coverage	22
Results	22
Cover properties	22
Waiver of Functional Coverage	23
5.6. Formal Verification	24
5.6.1. Gateway	24
5.6.2. Register Control	24
5.6.3. Request Handler	25

5.6.4. Target Notifier	25
5.6.5. AXI5 Lite Read Subordinate	25
5.6.6. AXI5 Lite Write Manager	25
5.6.7. AXI5 Lite Write Subordinate	26
6. Appendix	27
6.1. Integration Guide	27
6.2. Programming Guide	30
6.2.1. TT-APLIC Control Region Initialization	30
6.2.2. Additional information.	31
6.3. aplic_control_region_m address map	33
6.3.1. domaincfg register	39
BE field	39
DM field	39
RSVDZ1 field	39
IE field	39
RSVDZ0 field	39
6.3.2. sourcecfg register	39
CI_L_SM field	40
CI_H field	40
D field	40
6.3.3. mmsiaddrcfg register	40
LBPPN field	40
6.3.4. mmsiaddrcfg register	40
HBPPN field	41
LHXW field	41
HHXW field	41
LHXS field	41
HHXS field	41
L field	41
6.3.5. smsiaddrcfg register	41
LBPPN field	41
6.3.6. smsiaddrcfg register	41
HBPPN field	42
LHXS field	42
6.3.7. setip register	42
RDDATA field	42
WRDATA field	42
6.3.8. setipnum register	42
RDDATA field	43
WRDATA field	43
6.3.9. in_clrip register	43
RDDATA field	43
WRDATA field	43
6.3.10. clripnum register	43
RDDATA field	43
WRDATA field	43
6.3.11. setie register	44
RDDATA field	44
WRDATA field	44
6.3.12. setienum register	44
RDDATA field	44
WRDATA field	44
6.3.13. clrie register	44
RDDATA field	45
WRDATA field	45

6.3.14. clrienum register	45
RDDATA field	45
WRDATA field	45
6.3.15. setipnum_le register	45
RDDATA field	45
WRDATA field	45
6.3.16. setipnum_be register	46
RDDATA field	46
6.3.17. genmsi register	46
EIID field	46
Busy field	46
HI field	46
6.3.18. target register	46
EIID field	47
GI field	47
HI field	47
6.4. aplic_control_region_s address map	48
6.4.1. domaincfg register	53
BE field	54
DM field	54
RSVDZ1 field	54
IE field	54
RSVDZO field	54
6.4.2. sourcecfg register	54
CI_L_SM field	54
CI_H field	55
D field	55
6.4.3. mmsiaddrcfg register	55
LBPPN field	55
6.4.4. mmsiaddrcfgh register	55
HBPPN field	56
LHXW field	56
HHXW field	56
LHXS field	56
HHXS field	56
L field	56
6.4.5. smsiaddrcfg register	56
LBPPN field	56
6.4.6. smsiaddrcfgh register	56
HBPPN field	57
LHXS field	57
6.4.7. setip register	57
RDDATA field	57
WRDATA field	57
6.4.8. setipnum register	57
RDDATA field	57
WRDATA field	57
6.4.9. in_clrip register	58
RDDATA field	58
WRDATA field	58
6.4.10. clripnum register	58
RDDATA field	58
WRDATA field	58
6.4.11. setie register	58
RDDATA field	59

WRDATA field	59
6.4.12. setienum register	59
RDDATA field	59
WRDATA field	59
6.4.13. clrie register	59
RDDATA field	60
WRDATA field	60
6.4.14. clrienum register	60
RDDATA field	60
WRDATA field	60
6.4.15. setipnum_le register	60
RDDATA field	60
WRDATA field	60
6.4.16. setipnum_be register	60
RDDATA field	61
6.4.17. genmsi register	61
EIID field	61
Busy field	61
HI field	61
6.4.18. target register	61
EIID field	62
GI field	62
HI field	62
7. Revision History	63

Chapter 1. Overview

This document describes the Tenstorrent Advanced Platform-Level Interrupt Controller (TT-APLIC) specifications. The TT-APLIC handles external controller interrupts in the RISC-V ecosystem. The TT-APLIC also converts wired interrupts into MSI and notifies the IMSIC of interrupt occurrences in a RISC-V system with Harts that have IMSIC.

1.1. Implementation Notes

- **[Important]** The TT-APLIC may ignore incoming interrupts unless the IRQ signal is held in a particular state for 6 cycles or more.
 - The TT-APLIC ignores the IRQ signal during CSR write operations.
- The TT-APLIC supports only **two domains**.
 - Machine-level (M-domain). This is a root domain.
 - Supervisor-level (S-domain). This is a child of the M-domain.
- The TT-APLIC supports **MSI delivery mode** but not direct delivery mode.
- The **MSI address is not programmable** because it is determined by Verilog parameter..
- **Active-low reset** signal.
- The TT-APLIC has AXI subordinate ports for Read/Write requests to CSRs.
 - It asserts awready and wready once awvalid and wvalid are asserted.
 - It asserts arready when arvalid is asserted.
 - **The TT-APLIC ignores a write request if the wstrb is anything other than 4'b1111.**
- The TT-APLIC has AXI manager ports for MSI forwarding.
 - bready is set to 1.
- The TT-APLIC **ignores a write request** to a field with **illegal values**.
- This is single-clock domain.
- The TT-APLIC has **synchronous reset**. Please assert reset by one cycle to initialize the TT-APLIC.
- The following registers are **read-only**; their values cannot be changed:
 - MMSIADDRCFG
 - MMSIADDRCFGH
 - SMSIADDRCFG
 - SMSIADDRCFGH
 - SETIPNUM_BE

1.2. Repositories.

APLIC Specification

aus-gitlab.local.tenstorrent.com/arch/risc-v-aplic-spec

APLIC RTL

aus-gitlab.local.tenstorrent.com/soc/aplic

APLIC Model

aus-gitlab.local.tenstorrent.com/riscv/aplic

RISC-V Advanced Interrupt Architecture (AIA)

github.com/riscv/riscv-aia

1.3. TT-APLIC Functionality

The TT-APLIC converts incoming wired interrupts to corresponding MSIs. It includes a software-visible register to control interrupt processing. For example, the software can specify the trigger condition of the MSI transfer (e.g., rising edge, falling edge, ignore, etc...) using a **sourcecfg** register and the MSI destination using a **target** register. The MSI address is compatible with IMSIC defined in Advanced Interrupt Architecture specification.

- The TT-APLIC has CSRs to define its behavior (e.g., **sourcecfg**, **target**)
- TT-APLIC receives wired interrupts and registers them in "pending bits."
- TT-APLIC converts interrupts registered in the "pending bits" into MSI and send them to the IMSIC of harts.

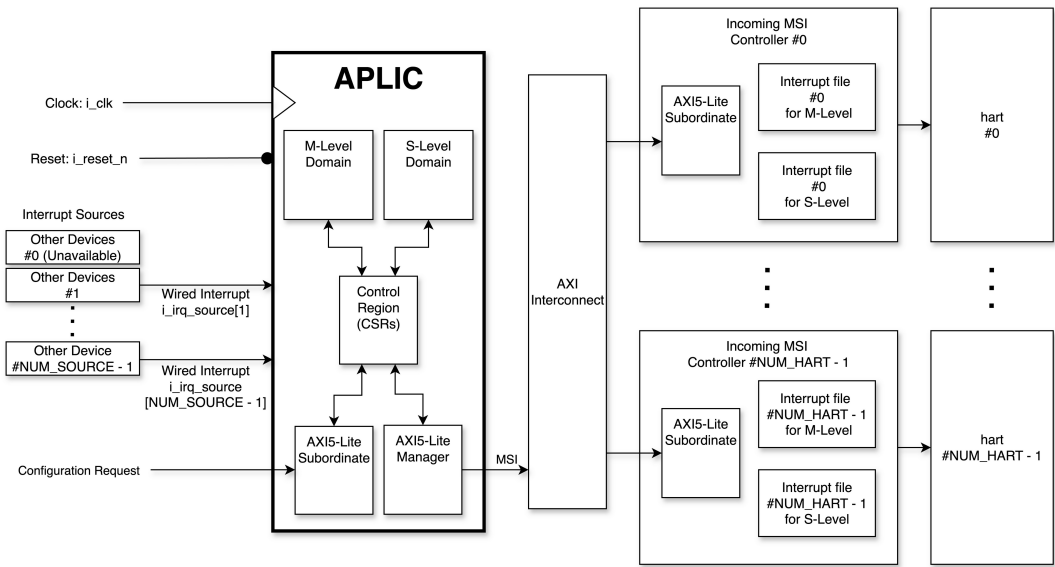


Figure 1. TT-APLIC Overview

1.4. Versions of Source Code and Tool

Category	Item	Branch/Version
Code	APLIC RTL	Branch: main
		Commit ID: a916c4ff11470c9eaf8cf1327133bda039e9e314
	APLIC Model	Branch: sminato/dev
		Commit ID: 3294e3efbb0a9ea4fc48db08aaaf5a5dda0737a7
Build Tool	Peakrdl	1.2.3
	python	3.9.18
	pandoc	3.6.2
Verification Tool	VCS, Verdi, URG	V-2023.12-SP2-5
	Jasper Gold	2024.06 FCS 64 bits

Table 1. Source files and Tool

1.5. Verification Result Summary

Item	Detail
Verification Date	2025/03/31
Smoke Test Result	Passed
Random Test Result	Passed
Random Test Coverage (Line, Branch, Toggle, Condition)	100% (Waiver is applied.)
Random Test Coverage (Function)	100% (Waiver is applied.)
Formal Verification	Passed

Table 2. Verification Result Summary

Chapter 2. Interface Specifications

2.1. Parameters

Each System Verilog module parameter uses the following values:

Parameter name	Width	Default value	Description
NUM_SOURCE	32	128 (0 - 1023)	Number of incoming interrupt wires
NUM_HART	32	128 (0 - 16383)	Number of supported harts.
NUM_GEILEN	32	5 (0 - 64)	Number of supported guest index from the S-mode interrupt domain (required by the Server SoC Spec)
CFG_AWID_WIDTH	32	10	AWID width for AXI subordinate port
CFG_ARID_WIDTH	32	10	ARID width for AXI subordinate port
CFG_ADDR_BASE_M	56	0x00_0000_1080_0000	M-level domain configuration register address base
CFG_ADDR_BASE_S	56	0x00_0000_2080_0000	S-level domain configuration register address base
CFG_ADDR_MASK_M	56	0x0F_FFFF_F00F_C000	M-level domain configuration register address mask
CFG_ADDR_MASK_S	56	0x0F_FFFF_F00F_C000	S-level domain configuration register address mask
MSI_BASE_ADDR_M	56	0x00_0000_4000_0000	M-level interrupt MSI base address
MSI_BASE_ADDR_S	56	0x00_0000_4400_0000	S-level interrupt MSI base address

Table 3. TT-APLIC implementation parameters

2.2. Input/Output Ports

The TT-APLIC has the following input ports:

Functionality	Signal name	Width	Description
Clock	i_clk	1	Clock signal. The TT-APLIC core has a single-clock domain.
Reset	i_reset_n	1	Reset signal (active-low, synchronous reset). Assert reset by one cycle to initialize the TT-APLIC.
Interrupt Source	i_irq_source[NUM_SOURCE-1:1]	1	Interrupt sources (Interrupt source 0 is unavailable.)

Table 4. Input Ports

Functionality	Signal name	Width	Description
AWVALID	i_cfg_s0_awvalid	1	
AWREADY	o_cfg_s0_awready	1	
AWID	i_cfg_s0_awid	CFG_AWID_WIDTH	
AWADDR	i_cfg_s0_awaddr	56	
WVALID	i_cfg_s0_wvalid	1	
WREADY	o_cfg_s0_wready	1	
WDATA	i_cfg_s0_wdata	32	
WSTRB	i_cfg_s0_wstrb	4	i_cfg_s0_wstrb[3:0] != 1111 return DECERR
BVALID	o_cfg_s0_bvalid	1	
BREADY	i_cfg_s0_bready	1	
BID	o_cfg_s0_bid	CFG_AWID_WIDTH	
BRESP	o_cfg_s0_bresp	2	
ARVALID	i_cfg_s0_arvalid	1	
ARREADY	o_cfg_s0_arready	1	
ARID	i_cfg_s0_arid	CFG_ARID_WIDTH	
ARADDR	i_cfg_s0_araddr	56	
RVALID	o_cfg_s0_rvalid	1	

Functionality	Signal name	Width	Description
RREADY	i_cfg_s0_rready	1	
RID	o_cfg_s0_rid	CFG_ARID_WIDTH	
RDATA	o_cfg_s0_rdata	32	
RRESP	o_cfg_s0_rresp	2	

Table 5. AXI subordinate ports (AXI Interface to set config registers (AXI5-Lite))

Functionality	Signal name	Width	Description
AWVALID	o_msi_m0_awvalid	1	
AWREADY	i_msi_m0_awready	1	
AWADDR	o_msi_m0_awaddr	56	
AWPROT	o_msi_m0_awprot	3	always 000
WVALID	o_msi_m0_wvalid	1	
WREADY	i_msi_m0_wready	1	
WDATA	o_msi_m0_wdata	32	
BVALID	i_msi_m0_bvalid	1	
BREADY	o_msi_m0_bready	1	
BRESP	i_msi_m0_bresp	2	
ARVALID	o_msi_m0_arvalid	1	always 0
ARREADY	i_msi_m0_arready	1	unused
ARADDR	o_msi_m0_araddr	56	always 0
ARPROT	o_msi_m0_arprot	3	always 0
RVALID	i_msi_m0_rvalid	1	unused
RREADY	o_msi_m0_rready	1	always 1
RDATA	i_msi_m0_rdata	32	unused
RRESP	i_msi_m0_rresp	2	unused

Table 6. AXI manager ports (AXI Interface to send MSI to RISC-V harts (AXI5-Lite))

Chapter 3. Architecture

3.1. Internal TT-APLIC Structure

Figure 2 shows the internal structure of the TT-APLIC. Please see Section 1.1 for important implementation-specific features.

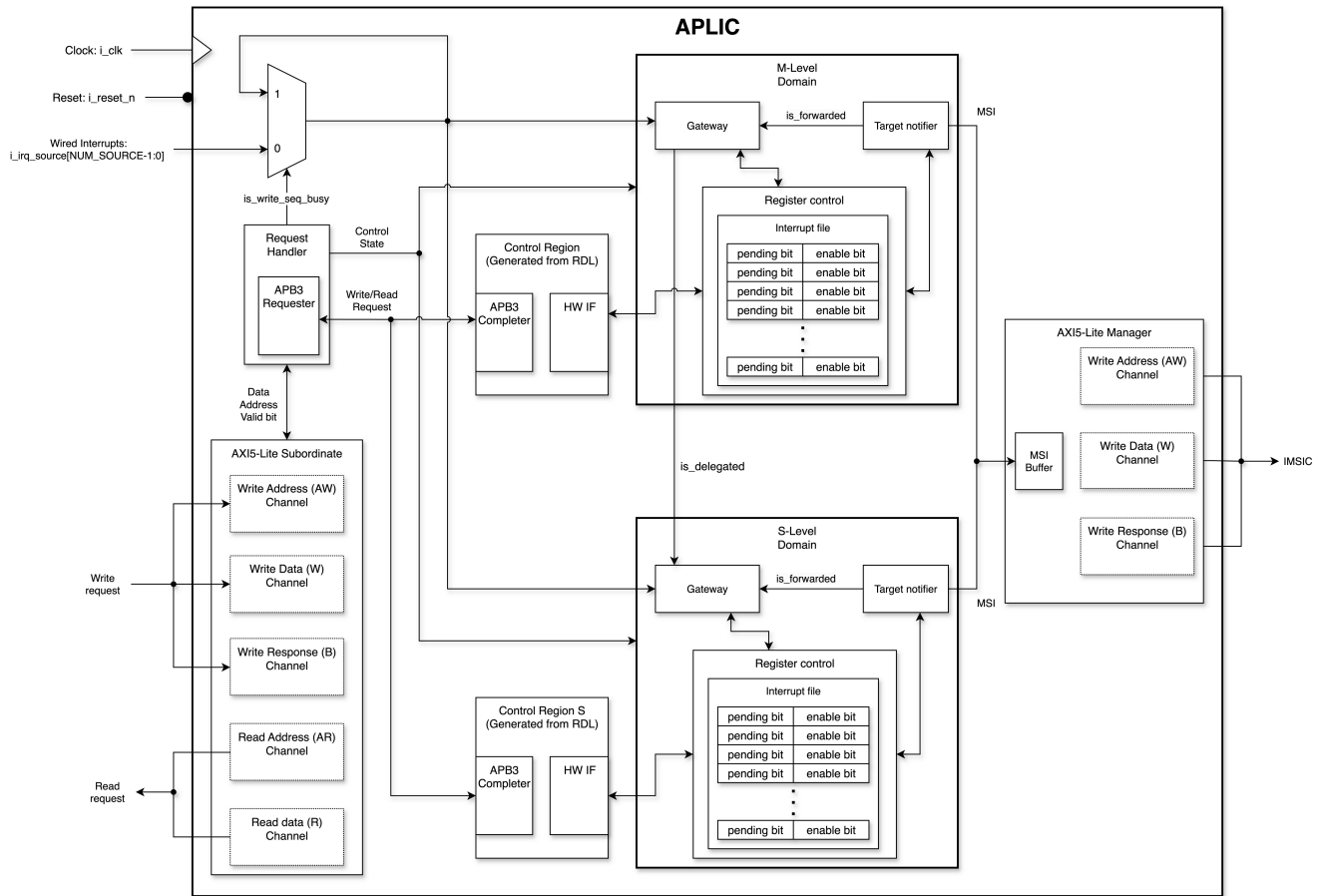


Figure 2. Internal TT-APLIC

[Important] The TT-APLIC may ignore the incoming interrupts unless the IRQ signal is kept in a given state for 6 cycles or more. The TT-APLIC ignores the IRQ signal for up to 5 cycles during CSR write operations.

An individual interrupt domain has the following components for each interrupt source at the TT-APLIC: ^[1]

Source configuration

This determines whether the specific source is active in the domain and, if so, how the incoming wire is to be interpreted, such as level-sensitive or edge-sensitive. Source configuration controls any delegation to a child domain for a source that is inactive in the domain.

Interrupt-pending and interrupt-enable bits

- Active source: The pending bit records an interrupt that arrived and has not yet been signaled or forwarded, while the interrupt-enable bit determines whether interrupts from this source should be delivered or remain pending.
- Inactive source: These two bits are read-only zeros.

Target selection

For an active source, target selection determines the hart to receive the interrupt and the new interrupt identity when forwarding as an MSI.

Each interrupt domain has the following features:

- Control Region
- Gateway
- Register Control
- Target Notifier

3.1.1. Control Region

Each TT-APLIC-supported interrupt domain has a dedicated memory-mapped control region (Control Region) for managing interrupts in that domain. The System RDL defines its register map. The TT-APLIC uses the RTL converted from System RDL by PeakRDL-regblock. The RTL generated by PeakRDL-regblock has the block architecture shown in Figure 3. (This is from PeakRDL-regblock: peakrdl-regblock.readthedocs.io/en/latest/architecture.html)

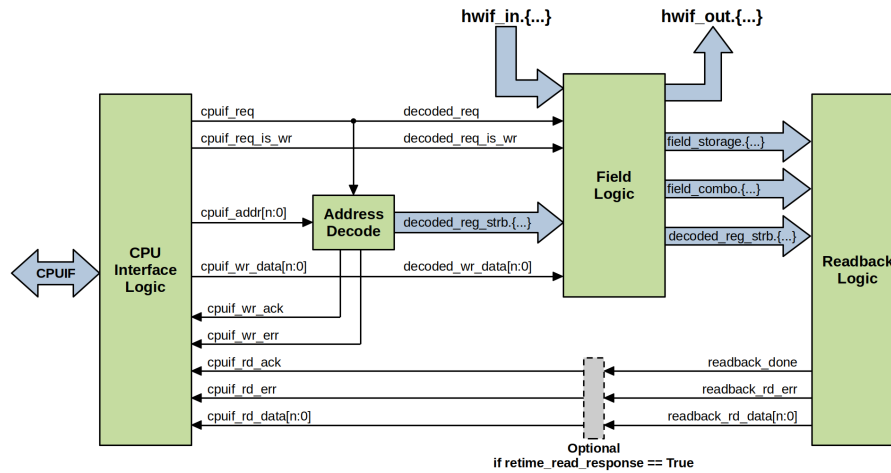


Figure 3. Register Block Architecture

The CUIF is an APB interface in this TT-APLIC implementation that is connected to the Section 3.1.2. Table 7 shows the addresses and reset values. Please see Section 6.3 and Section 6.4 for the detail.

Offset	Name	Reset value	Description
0x0000	domaincfg	0x8000_0004	IE = 0, DM = 1 (MSI delivery mode), BE = 0 (Little endian)
0x0004	sourcecfg[0]	0x0	Reset value is zero, and the index starts from 0. sourcecfg[i] controls i_irq_source[i+1].
...	...	...	...
0x01FC	sourcecfg[126]	0x0	Reset value is zero, and the index starts from 0. sourcecfg[i] controls i_irq_source[i+1].
0x1BC0	mmsiaddrcfg	0x0	Read-only.
0x1BC4	mmsiaddrcfggh	0x8000_0000	Read-only.
0x1BC8	smsiaddrcfg	0x0	Read-only.
0x1BCC	smsiaddrcfggh	0x0	Read-only.
0x1C00	setip[0]	0x0	Reset value is zero.
...	...	...	...
0x1C0C	setip[3]	0x0	Reset value is zero.
0x1CDC	setipnum	0x0	Reset value is zero.
0x1D00	in_clrip[0]	0x0	Reset value is zero.
...	...	...	...
0x1D0C	in_clrip[3]	0x0	Reset value is zero.
0x1DDC	clripnum	0x0	Reset value is zero.
0x1E00	setie[0]	0x0	Reset value is zero.
...	...	...	...
0x1E0C	setie[3]	0x0	Reset value is zero.
0x1EDC	setienum	0x0	Reset value is zero.
0x1F00	clrie[0]	0x0	Reset value is zero.
...	...	...	...
0x1F0C	clrie[3]	0x0	Reset value is zero.
0x1FDC	clrienum	0x0	Reset value is zero.

Offset	Name	Reset value	Description
0x2000	setipnum_le	0x0	Reset value is zero.
0x2004	setipnum_be	0x0	Reset value is zero.
0x3000	genmsi	0x0	Reset value is zero.
0x3004	target[0]	0x0	Reset value is zero, and the index starts from 0. target[i] controls i_irq_source [i+1].
...	...	...	...
0x31FC	target[126]	0x0	Reset value is zero, and the index starts from 0. target[i] controls i_irq_source [i+1].

Table 7. Register Addresses and Reset Values

3.1.2. Request Handler

This module has the following features.

- Decode address in write/read request.
 - Capture the write/read request from the AXI subordinate.
 - Select a domain to access by decoding the address.
 - Calculate the destination index in a control region.
- Check if the write request value is legal or not. If not, then the write value is masked and restored to the previous value stored in the control region.
- Handle APB transaction.
 - Send write/read requests to the control region using the APB protocol.
 - Receive write/read responses from the control region and send them to the AXI subordinate.

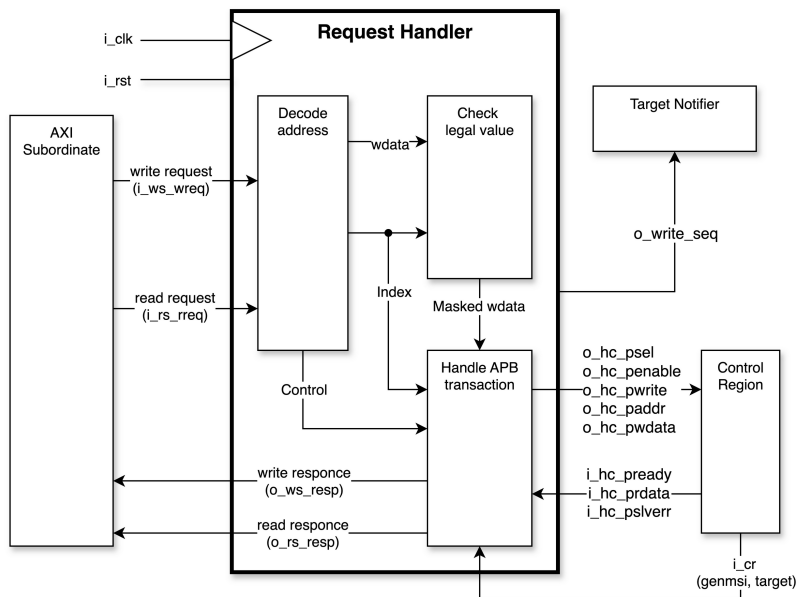


Figure 4. Request Handler

3.1.3. Gateway

This module has the following features.

- Delegates each interrupt source to a child domain, if that interrupt is set to be delegated in each sourcecfg.
- Rectifies each interrupt source using the source mode set in each sourcecfg.SM.
- Detects the rising edge of each interrupt source and notifies Register Control.

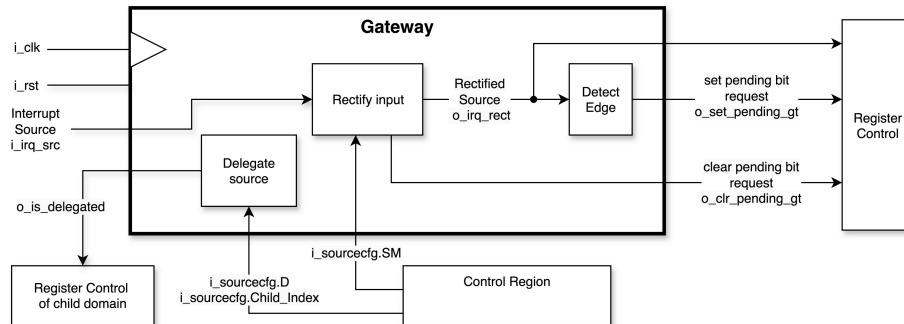


Figure 5. Gateway

3.1.4. Register Control

The Register Control controls and updates the registers in the control region. It has one interrupt-pending bit and one interrupt-enable bit for every interrupt source, and includes the following key features:

- Controls and updates the registers in the control region except **genmsi**.
 - Clears and locks **sourcecfg** if an interrupt source is not delegated to the domain.
 - Sets **setip** to the value of pending bits.
 - Sets **in_clrip** to the value of rectified interrupts.
 - Sets **setie** to the value of enable bits.
 - Clears and locks **target** if an interrupt source is inactive in the domain.
- Updates the interrupt-pending bit.
 - Clears a pending bit if:
 - an interrupt source is inactive in the domain.
 - a rectified interrupt is low and the source is configured to level-sensitive.
 - a write request to clrip or clripnum is claimed.
 - Sets a pending bit if:
 - a new interrupt is asserted.
 - a write request to setip, setipnum, or setipnum_le is claimed.
 - a pending bit has been already set.
- Updates the interrupt-enable bit.
 - Clears a enable bit if:
 - an interrupt source is inactive in the domain.
 - a write request to clrie or clrienum is claimed.
 - Sets a enable bit if:
 - a write request to setie or setienum is claimed.
 - a enable bit has been already set.

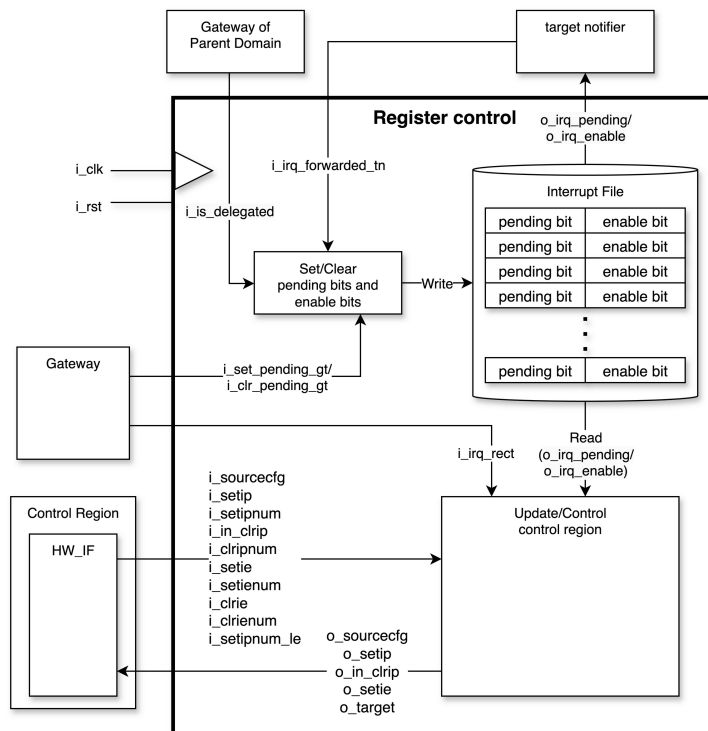


Figure 6. Register Control

3.1.5. Target Notifier

The Target Notifier checks for new or pending interrupts via the Register Control and, if any, creates an MSI and sends it to the AXI5 Lite manager. It includes the following features:

- Calculate MSI address
 - Reads the **target** register value from the control region and calculates the MSI address for each interrupt source.
 - Reads the **genmsi** register value from the control region and calculates the address of the extempore MSI.
- Selects "one" pending and enable interrupt to send by:
 - Reading the pending bits and enabling bits output from register control.
 - Selecting and enabling one pending interrupt.
- Controls GENMSI.
 - Detects write accesses to **genmsi**, and if any, sets the **genmsi** busy bit to one and locks **genmsi**.
- Send MSI to the AXI5 lite manager.
 - Send MSI for the selected interrupt to the AXI5 lite manager.
 - If **genmsi.Busy** is set, sends extempore MSI generated from **genmsi** at first.

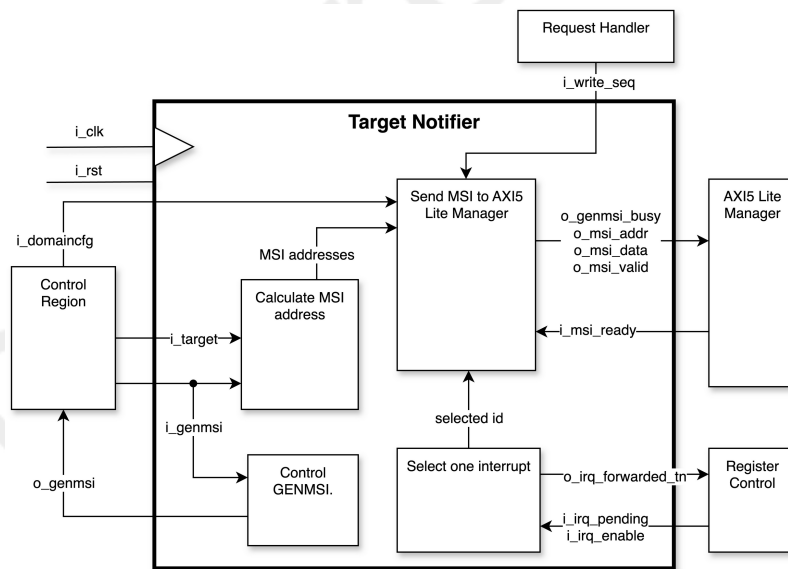


Figure 7. Target Notifier

3.2. Function

- Write request operation: A write request occurs when both AWVALID and WVALID are asserted simultaneously.
- Operation When a read request occurs
- Interrupt assertion operation:
 - Interrupts that occur up to the 4th cycle after a register write request (cycle 0) are processed based on the pre-update register values and saved in the pending bit.** Interrupts that occur 5 or more cycles after the register write request are processed based on the updated register values and saved in the pending bit.
- MSI forwarding: After being registered in the Pending bit, specify the minimum number of cycles required for output from the TT-APLIC.

[1] aplic-doc-org, The RISC-V Advanced Interrupt Architecture Version 1.0

Chapter 4. Timing Chart

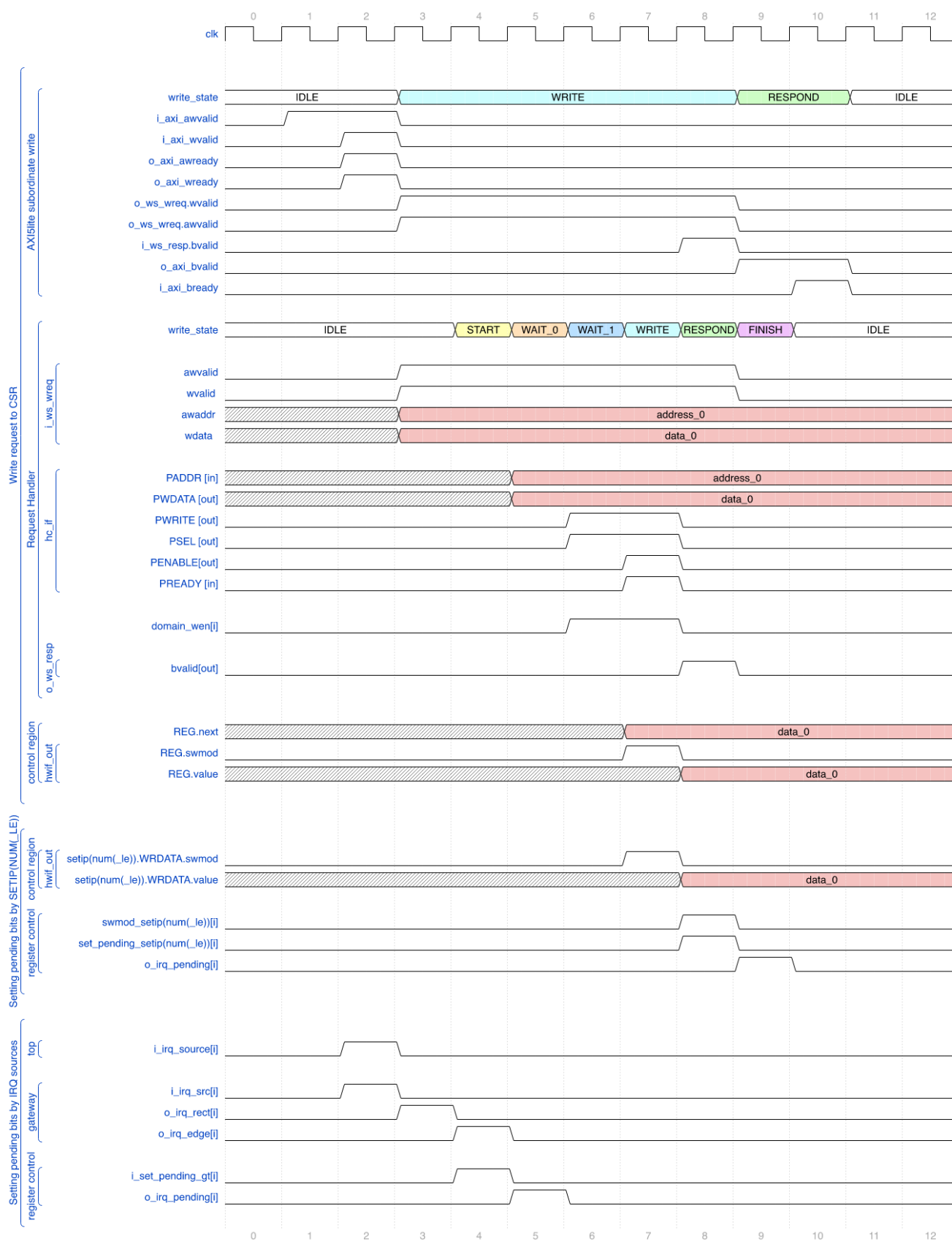


Figure 8. TT-APLIC Timing chart

Chapter 5. Verification

5.1. Verification Environment

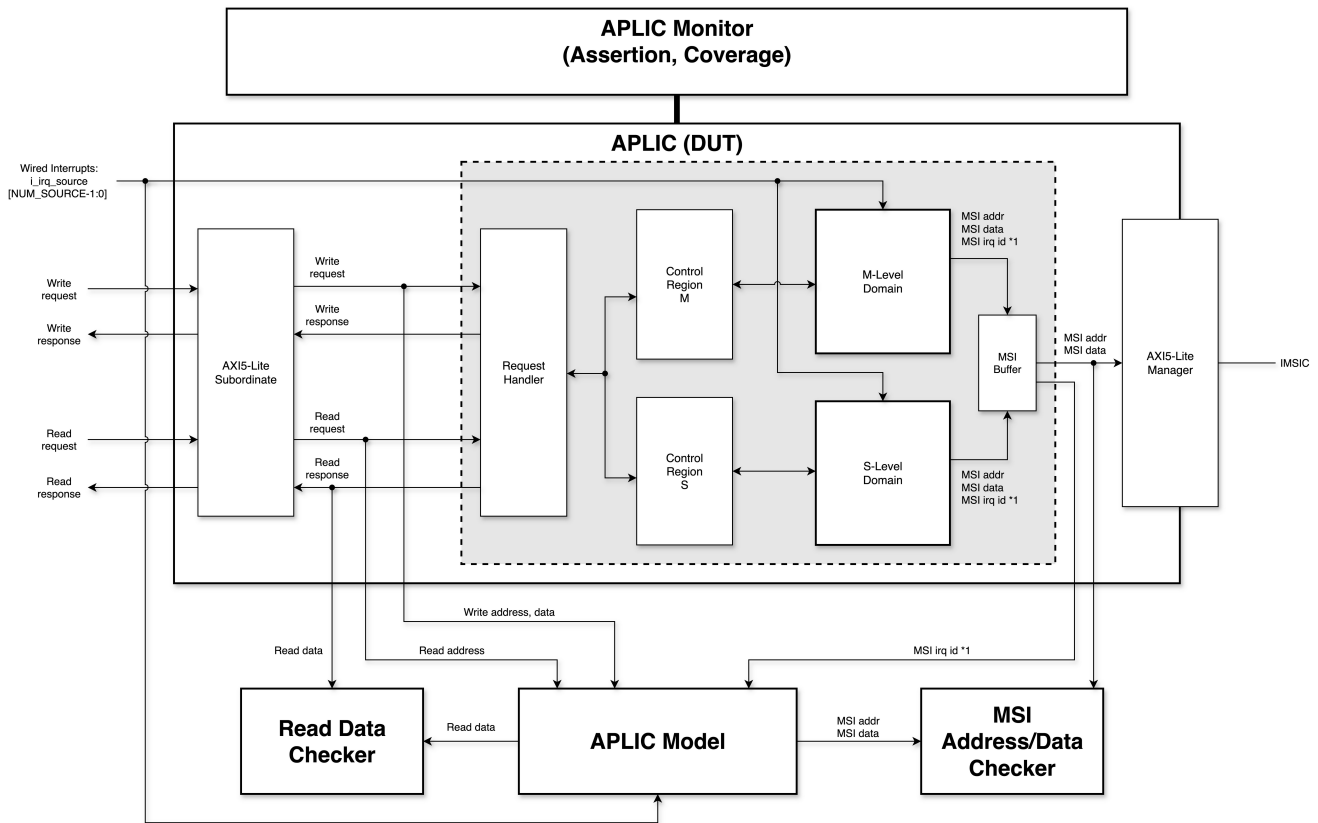


Figure 9. Verification Environment

- For tests, the DUT (TT-APLIC) is connected to the TT-APLIC functional model (the TT-APLIC model). TT-APLIC model APIs are invoked to check the consistency of values when an event occurs.
- If the state of a wired interrupt input to the RTL changes, then the same change is also input to the TT-APLIC model.
- If write access is detected in tests, then the test automatically calls API and writes the same value to the register in the TT-APLIC model.
- If read access is detected in tests, then the test automatically calls the API, reads the value of the same address in the TT-APLIC model, and verifies that it matches the value in the tests.
- If the RTL sends an MSI, then the interrupt number of the interrupt that triggered the MSI (0 in the case of genmsi) is also notified to the TT-APLIC model. The model then verifies whether the MSI should have been sent and whether its data and address are correct.

5.2. Common Smoke & Random Test Initialization Process

5.2.1. Initialization Procedure

1. Generate Clock
2. Reset Test values
 - a. IRQ requests
 - b. Temporary signals
3. Reset TT-APLIC
 - a. Reset signal `i_reset_n` is set to 1 and held for one cycle.
 - b. Reset signal `i_reset_n` is set to 0 and held for one cycle.
 - c. Reset signal `i_reset_n` is set to 1 and held.

4. The following registers are initialized. See initial values in sections below.

- a. **domaincfg**
- b. **sourcecfg**
- c. **target**
- d. **setie**

Note

This only applies to smoke and random tests. It is not intended formal verification.

5.2.2. Initial domaincfg Values for Testing

For smoke tests, the value of **domaincfg** is initialized to **0x8000_0104**. This means IE = 1, DM = 1, BE = 0.

5.2.3. Initial sourcecfg Values for Testing

For smoke tests, a **sourcecfg** is initialized in the following order, starting from index 0.

Initializing order	Sourcecfg in M-domain	Sourcecfg in S-domain
0	INACTIVE	N/A (read-only zero)
1	DETACHED	N/A (read-only zero)
2	EDGE1	N/A (read-only zero)
3	EDGE0	N/A (read-only zero)
4	LEVEL1	N/A (read-only zero)
5	LEVEL0	N/A (read-only zero)
6	Delegated to S-domain	INACTIVE
7	Delegated to S-domain	DETACHED
8	Delegated to S-domain	EDGE1
9	Delegated to S-domain	EDGE0
10	Delegated to S-domain	LEVEL1
11	Delegated to S-domain	LEVEL0

Table 8. Initial values for sourcecfg in each domain.

For example, if the remainder when the interrupt number of a **sourcecfg** is divided by 12 equals 6, then then source is delegated to an S-domain and treated as an INACTIVE source.

5.2.4. Initial target Values for Testing

For smoke tests, the value of **target** is initialized to the following value if it is not inactive in the domain.

Field	Initial value
target[i].HI	i (the same as its interrupt number)
target[i].GI	i (the same as its interrupt number)
target[i].EIID	i (the same as its interrupt number)

5.2.5. Initial setie Values for Testing

For smoke tests, all **setie** bits are initialized to one if it is not inactive in the domain. This means all interrupts except inactive ones are enabled.

5.3. Smoke Test

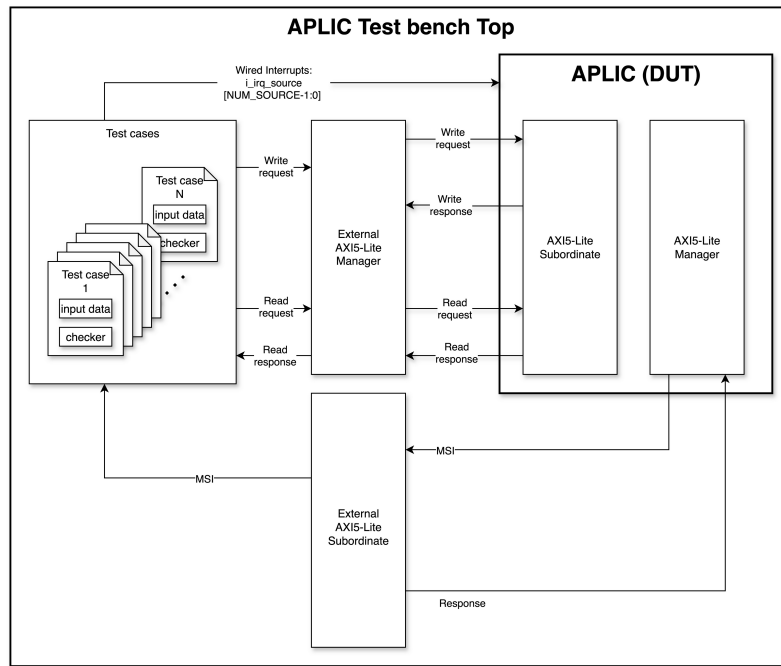


Figure 10. APLIC Smoke Test Environment

In the smoke test, an external AXI5 Lite Manager and Subordinate are connected to the APLIC. The test bench inputs interrupts to the APLIC (DUT) and sends write/read requests and responses to the AXI5 Lite Manager/Subordinate.

5.3.1. test 1: Updating and Reading configuration registers

Overview

- This test attempts to write to registers in the control regions.
- After completing each write operation, the test reads the registers to verify that legal values have been written and match the values from the TT-APLIC model. The target registers are:
 - domaincfg
 - sourcecfg
 - mmsiaddrcfg
 - mmsiaddrcfgh
 - smsiaddrcfg
 - smsiaddrcfgh
 - target
- After completing all write operations, the test reads all registers in the control region and verifies the that read values match the values from the TT-APLIC model.

Test procedure

1. Initialize the test according to the procedure in [Section 5.2.1](#).
2. Write a random value to a register.
3. Read the value of the register.
4. Repeat steps 2. and 3. for domaincfg, sourcecfg, mmsiaddrcfg, mmsiaddrcfgh, smsiaddrcfg, smsiaddrcfgh, and target.
5. Read all registers.

(If the tests detect a write access, then the test automatically calls an API and writes the same value to the register in the TT-APLIC model.) (If the tests detect a read access, then the test automatically calls an API, reads the value of the same address in the TT-APLIC model, and verify that it matches the value in the tests.)

Expected results and current status

- Read values matches the expected values defined in the tests = PASSED
- Read values matches the values read from the TT-APLIC model = PASSED

5.3.2. test 2-5: Forwarding MSI with Level/Edge-Sensitive and Active High/Low Interrupts

Overview

- This test attempts to send MSIs using wired interrupts.
- If an MSI is sent in RTL, then the test determines whether or not to send it using the TT-APLIC model.
- The test also verifies that the MSI address and data match those of the MSI from the TT-APLIC model.

Test procedure

1. Initialize the test according to the procedure in [Section 5.2.1](#).
2. Set **sourcecfg**, **target**, and **setie** of an interrupt number for the M-domain.
3. Assert wired interrupt of an interrupt number.
4. Check MSI.
5. Reset wired interrupt of an interrupt number.
6. Set **sourcecfg** and delegate an interrupt of an interrupt number.
7. Set **sourcecfg**, **target**, and **setie** of the interrupt number for the S-domain.
8. Assert wired interrupt of an interrupt number.
9. Check the MSI.

Expected results and current status

- When a MSI is sent, the model will verify that it can be sent, and then send it = PASSED
- The address and data of a MSI matches those of the MSI from the TT-APLIC model = PASSED
- Ensure that a MSI is sent within a finite number of cycles = PASSED

5.3.3. test 6,7: Handling Multiple Interrupts in the one/multiple domain(s)

Overview

- This test attempts to send MSIs using multiple wired interrupts.
- If an MSI is sent in RTL, then the test determines whether or not it should be sent using the TT-APLIC model.
- The test verifies that the MSI address and data match those of the MSI from the TT-APLIC model.
- The test creates a list of MSIs that should be sent, and then verifies that all of them are actually sent.

Test procedure

1. Initialize the test according to the procedure in [Section 5.2.1](#).
2. Set **sourcecfg**, **target**, and **setie** for the M-domain and S-domain.
3. Assert multiple wired interrupts.
4. Check MSIs.

Expected results and current status

- The model verifies that an MSI can be sent, and then sends it = PASSED
- The MSI address and data matches those of the MSI from the TT-APLIC model = PASSED
- Ensure that a MSI is sent within a finite number of cycles = PASSED

5.3.4. test 8,9: Handling Simultaneous Interrupts and Config Requests in M/S-domain

Overview

- This test attempts to send MSIs while continuous write requests are being processed.
- Check that at least one MSI is sent between write requests.
- If MSI is sent in RTL, then the test determines whether or not it should be sent using the TT-APLIC model.
- The test verifies that the MSI address and data of MSI matches those of the MSI from the TT-APLIC model.

Test procedure

1. Initialize the test according to the procedure in [Section 5.2.1](#).
2. Set **sourcecfg**, **target**, and **setie** for the M-domain and S-domain.
3. Start a new separate thread that repeatedly writes new values to **target**.
4. Assert a wired interrupt.

5. Check MSIs.

Expected results and current status

- Check that at least one MSI is sent between write requests = PASSED
- If a MSI is sent in RTL, then the test determines whether or not to send it using the TT-APLIC model = PASSED
- The test verifies that the MSI address and data matches those of the MSI from the TT-APLIC model = PASSED

5.3.5. test 10,11: Handling genmsi in M/S-domain

Overview

- This test attempts to send extempore MSIs using **genmsi**.
- Check that at least one extempore MSI is sent.
- If the MSI is sent in RTL, then the test determines whether or not to send it using the TT-APLIC model.
- The test verifies that the MSI address and data of MSI matches those of the MSI from TT-APLIC model.

Test procedure

1. Initialize the test according to the procedure in [Section 5.2.1](#).
2. Set **sourcecfg**, **target**, and **setie** for M-domain and S-domain.
3. Create and execute a separate thread that repeatedly writes new values to **genmsi**.
4. Check MSIs.
5. Assert a wired interrupt.
6. Check MSIs.

Expected results and current status

- Check that at least one extempore MSI is sent = PASSED
- If an MSI is sent in RTL, then the test determines whether or not it should be sent using the TT-APLIC model = PASSED
- The test verifies that the MSI address and data matches those of the MSI from the TT-APLIC model = PASSED

5.3.6. test 13-18: Forwarding msi Using setip(num(_le)) in M/S-domain

Overview

- This test attempts to send MSIs using **setip(num(_le))**.
- Check that at least one MSI from **setip(num(_le))** is sent.
- If a MSI is sent in RTL, then the test determines whether or not it should be sent using the TT-APLIC model.
- The test verifies that the MSI address and data matches those of the MSI from the TT-APLIC model.

Test procedure

1. Initialize the test according to the procedure in [Section 5.2.1](#).
2. Set **sourcecfg**, **target**, and **setie** for the M-domain and S-domain.
3. Set **setip(num(_le))** for the M-domain or S-domain.
4. Check MSIs.

Expected results and current status

- Check that at least one MSI from **setip(num(_le))** is sent = PASSED
- If a MSI is sent in RTL, then the test determines whether or not it should be sent using the TT-APLIC model = PASSED
- The test verifies that the MSI address and data matches those of the MSI from the TT-APLIC model = PASSED

5.3.7. test 19: Stopping MSI Forwarding in disabled domain

Overview

- This test attempts to stop sending MSIs using **domaincfg.IE**.
- Check that no MSI is sent if **domaincfg.IE** and **genmsi.Busy** is set to 0.
- Check that an extempore MSI can be sent even if **domaincfg.IE** is set to 0.

Test procedure

1. Initialize the test according to the procedure in [Section 5.2.1](#).
2. Set **sourcecfg**, **target**, and **setie** for the M-domain and S-domain.

3. Assert a wired interrupt.
4. Check MSIs.
5. Set **domaincfg.IE** to 0.
6. Check that no MSI is sent.
7. Set **genmsi**.
8. Check **genmsi**.

Expected results and current status

- Check that no MSI is sent if **domaincfg.IE** and **genmsi.Busy** are not set to 0 = PASSED
- Check that an extempore MSI can be sent even if **domaincfg.IE** is set to 0 = PASSED

5.4. Random Test

Overview

- This test asserts random values to the TT-APLIC input port of every cycle and verifies that no unexpected errors occur.
- This uses the same assertions as those used in Formal Verification.
- This also uses the TT-APLIC model. The API is called whenever a write/read request occurs, interrupts are asserted, and an MSI is sent to check its behavior.

Input patterns

Input signal	Description
i_clk	Connected to a clock generator.
i_reset_n	Kept 1 after the initialization.
i_irq_source	Assert completely random value at every cycle.
i_cfg_s0_awaddr	Explained in Section Section 5.4.2 .
i_cfg_s0_wdata	Explained in Section Section 5.4.1 .
i_cfg_s0_wstrb	Explained in Section Section 5.4.3 .
i_cfg_s0_araddr	Explained in Section Section 5.4.2 .
Other AXI signals	Assert completely random value at every cycle.

5.4.1. Generating random data for read/write requests

One of the following data is selected for **i_cfg_s0_wdata** in each cycle.

Data can be selected	Description
0x0000_0000	All bits are 0.
0xFFFF_FFFF	All bits are 1.
A completely random data	A random address.
A random data for domaincfg	Write 0 or 1 to domaincfg.IE.
A random data for sourcecfg	Write 0 or 1 to sourcecfg[n].D and SM=INACTIVE, DETACHED, LEVEL1, LEVEL0, EDGE1 or EDGE0 to sourcecfg[n].SM
A random data for target and genmsi	Write illegal or legal value to target/genmsi.HI, (GI,) and EIID.
A random data for setipnum, clripnum, setienum and clienum	Write interrupt ID from 0 to NUM_SOURCE - 1

5.4.2. Generating random addresses for read/write requests

A random address is determined in each cycle as follows:

1. A base address is chosen from either CFG_ADDR_BASE_M or CFG_ADDR_BASE_S.
2. One of the following addresses is selected.

Address can be selected	Description
0x00_0000_0000_0000	All bits are 0.
0xFF_FFFF_FFFF_FFFF	All bits are 1.
A completely random address	A random address.
The base address + an random integer from 0 to 16,384	Within a control region.
The base address + 0x0000	domaincfg
The base address + 0x0004 + an random integer from 0 to NUM_SOURCE - 2	sourcecfg[1] - sourcecfg[NUM_SOURCE - 2]
The base address + 0x1BC0	mmsiaddrcfg
The base address + 0x1BC4	mmsiaddrcfg
The base address + 0x1BC8	smsiaddrcfg
The base address + 0x1BCC	smsiaddrcfg

Address can be selected	Description
The base address + 0x1C00 + an random integer from 0 to NUM_REG - 1	setip[0] - setip[NUM_REG - 1]
The base address + 0x1CDC	setipnum
The base address + 0x1D00 + an random integer from 0 to NUM_REG - 1	in_clrip[0] - in_clrip[NUM_REG - 1]
The base address + 0x1DDC	clripnum
The base address + 0x1E00 + an random integer from 0 to NUM_REG - 1	setie[0] - setie[NUM_REG - 1]
The base address + 0x1EDC	setienum
The base address + 0x1F00 + an random integer from 0 to NUM_REG - 1	clrie
The base address + 0x1FDC	clrienum
The base address + 0x2000	setipnum_le
The base address + 0x2004	setipnum_be
The base address + 0x3000	genmsi
The base address + 0x3004 + an random integer from 0 to NUM_SOURCE - 1	target[1] - target[NUM_SOURCE - 2]

Note: the following addresses have a lower probability of being selected compared to others. These are addresses for invalid or read-only registers.

- All bits are 0.
- All bits are 1.
- A random address.
- Within a control region.
- mmsiaddrcfg
- mmsiaddrcfg
- smsiaddrcfg
- smsiaddrcfg
- setipnum_be

5.4.3. Generating random data for wstrb

A random value for wstrb is selected in each cycle as follows: . One of the following value is selected.

value can be selected	Description
0b1111	A valid write strobe.
An random integer from 0 to 15	A random value.

5.5. Coverage Test

Overview

- Measured the coverage of the following criteria to evaluate the coverage of the Random Test:
 - Line coverage
 - Branch coverage
 - Condition coverage
 - Toggle coverage
- Achieved 100% coverage for each criterion by excluding unreachable lines, branches, conditions, and non-toggling bits from the evaluation:
- Confirmed the functional coverage to ensure valid inputs were provided during the random testing process.
- Changed NUM_SOURCE from 128 to 4 for the coverage test to prevent memory shortages during verification.

5.5.1. Results of Line, Branch, Condition, and Toggle Coverage.

Name	Score	Line	Toggle	Condition	Branch
aplic_model_tb_top	90.36%	98.13%	63.72%	99.64%	99.93%
aplic	90.41%	98.33%	63.72%	99.64%	99.95%
axiv_lite_write_m...	81.35%	100.00%	28.62%	96.77%	100.00%
s_domain	83.87%	100.00%	45.72%	89.76%	100.00%
target_notifier	76.76%	100.00%	19.56%	87.50%	100.00%
gateway	82.54%	100.00%	61.97%	68.18%	100.00%
register_control	94.02%	100.00%	78.94%	97.14%	100.00%
m_domain	84.30%	100.00%	43.81%	93.37%	100.00%
target_notifier	76.49%	100.00%	18.46%	87.50%	100.00%
gateway	85.87%	100.00%	66.20%	77.27%	100.00%
register_control	93.88%	100.00%	75.52%	100.00%	100.00%
request_handler	92.83%	100.00%	78.93%	92.39%	100.00%
control_region_m	93.66%	96.04%	78.87%	99.83%	99.89%
control_region_s	94.80%	100.00%	79.37%	99.83%	100.00%
axiv_lite_read_su...	95.40%	100.00%	84.83%	96.77%	100.00%
axiv_lite_write_su...	96.73%	100.00%	94.59%	92.31%	100.00%

Figure 11. The test coverage of before applying the waiver

Name	Score	Line	Toggle	Condition	Branch
aplic_model_tb_top	99.93%	99.73%	100.00%	99.99%	99.98%
aplic	100.00%	100.00%	100.00%	100.00%	100.00%
axiv_lite_write_m...	100.00%	100.00%	100.00%	100.00%	100.00%
s_domain	100.00%	100.00%	100.00%	100.00%	100.00%
target_notifier	100.00%	100.00%	100.00%	100.00%	100.00%
gateway	100.00%	100.00%	100.00%	100.00%	100.00%
register_control	100.00%	100.00%	100.00%	100.00%	100.00%
m_domain	100.00%	100.00%	100.00%	100.00%	100.00%
target_notifier	100.00%	100.00%	100.00%	100.00%	100.00%
gateway	100.00%	100.00%	100.00%	100.00%	100.00%
register_control	100.00%	100.00%	100.00%	100.00%	100.00%
request_handler	100.00%	100.00%	100.00%	100.00%	100.00%
control_region_m	100.00%	100.00%	100.00%	100.00%	100.00%
control_region_s	100.00%	100.00%	100.00%	100.00%	100.00%
axiv_lite_read_su...	100.00%	100.00%	100.00%	100.00%	100.00%
axiv_lite_write_su...	100.00%	100.00%	100.00%	100.00%	100.00%

Figure 12. The test coverage of after applying the waiver

5.5.2. Waiver for Line, Branch, Condition, and Toggle Coverage.

Line Coverage

- hwclr of sourcecfg in M-domain will never be set to 1.

Condition Coverage

- `o_msi_ready` will not be set to 1 when `i_msi_valid` is set to 0.
- Only one of `i_genmsi.HI.swmod` and `i_genmsi.EIID.swmod` will never be set to 1. They will be both 0 or both 1.
- `o_msi_valid` is always set to 1 when `o_genmsi.busy` is set to 1.
- No interrupt source will be delegated from S-domain because it does not have any child domain.
- An interrupt source will not be delegated to any domain other than S-domain.
- `i_sourcecfg.CI_L_SM` is always set to 0 when an interrupt source is delegated. (This is because the number of child domain is 1.)
- Since the first part of the OR expression evaluated to 1, the remaining expression was not evaluated.
- An exception is raised if `pready` does not become 1 in the next cycle after `penable` becomes 1, but such an exception was not observed.
- Only one of `i_ws_wreq.awvalid` and `i_ws_wreq.wvalid` will never be set to 1. They will be both 0 or both 1.
- An address always indicates the registers in M-domain or S-domain if `addr_offset` in request handler is valid.
- An address offset is never a negative value.
- The logic type is unsigned, so it never takes negative values.
- `is_in_domain_w` is always 1 when `wdata_sourcecfg.D1` is 1.
- `decoded_wr_biten` in control region is always '1'.
- `control_region` never returns any error for read/write requests.
- `cpuif_req_stall_rd` and `cpuif_req_stall_wr` is always '0'.
- `sourcecfg.SM.swwe` and `sourcecfg.D.swwe` is always set to 1 in M-domain.
- `sourcecfg.D` is always set to 0 in S-domain.
- `o_axi_arready` is never set to 1 if `i_axi_arvalid` is set to 0.
- `o_axi_awready` is never set to 1 if `i_axi_awvalid` is set to 0.
- `o_axi_wready` is never set to 1 if `i_axi_wvalid` is set to 0.
- Only one of `o_ws_wreq.wvalid` and `o_ws_wreq.awvalid` will never be set to 1. They will be both 0 or both 1.

Toggle coverage

- The number of interrupts has been set to 4 (including ID=0), so only the 1st to 3rd bits in `setip`, `in_clrip`, and `setie` are valid.
- `hwclr` and `swwe` of `sourcecfg` in M-domain will never be set.
- Only 7 bits are required for Hart Index since the maximum value of the Heart ID is 127.
- `sourcecfg.CI_H` (MSB of child index) is read-only 0, because the Child index never takes a value other than 0.
- `sourcecfg.D` will not be set in S-domain.
- The 3th to 5th bits of `target.GI` is nor required because `NUM_GEILEN` is set to 4.
- The control region does not return error: `hc_pslverr`.
- The read address channel and read data channel for MSI are not used.
- The bit 11, bits from 15 to 17 and bits from 25 to 30 of read values do not change. For the detail, please refer to "Block 1".
- The bits from 11 to 31 of MSI data do not change, because the length of the EIID field of `target` and `genmsi` registers are 11 bits.
- `o_msi_m0_bready` and `o_msi_m0_awprot` is always zero.

0 - always zero
1 - always one
X - zero or one

```
domaincfg    1000_0000_0000_0000_0000_000X_0000_0100
sourcecfg    0000_0000_0000_0000_0000_0X00_0000_0XXX
mmsiaddrcfg  1000_0000_0000_0000_0000_0000_0000_0000
mmsiaddrcfg  0000_0000_0000_0000_0000_0000_0000_0000
smsiaddrcfg  0000_0000_0000_0000_0000_0000_0000_0000
smsiaddrcfg  0000_0000_0000_0000_0000_0000_0000_0000
```

```

setip      0000_0000_0000_0000_0000_0000_0000_XXX0
setipnum   0000_0000_0000_0000_0000_0000_0000_0000
in_clrip   0000_0000_0000_0000_0000_0000_0000_XXX0
clripnum   0000_0000_0000_0000_0000_0000_0000_0000
setie      0000_0000_0000_0000_0000_0000_0000_XXX0
setienum   0000_0000_0000_0000_0000_0000_0000_0000
clrie      0000_0000_0000_0000_0000_0000_0000_0000
clrienum   0000_0000_0000_0000_0000_0000_0000_0000
setipnum_le 0000_0000_0000_0000_0000_0000_0000_0000
setipnum_be 0000_0000_0000_0000_0000_0000_0000_0000
genmsi     0000_000X_XXXX_XX00_000X_0XXX_XXXX_XXXX
target     0000_000X_XXXX_XX00_0XXX_0XXX_XXXX_XXXX

Total      X000_000X_XXXX_XX00_0XXX_0XXX_XXXX_XXXX

```

Block 1. The read values of each registers

- The bits from 0 to 11, bits from 15 to 17, bit 25, and bits from 27 to 55 of MSI addresses do not change. For the detail, please refer to "Block 2".

0 - always zero
1 - always one
X - zero or one

```

MSI_BASE_ADDR_M: ..._0100_0000_0000_0000_0000_0000_0000
MSI_BASE_ADDR_S: ..._0100_0100_0000_0000_0000_0000_0000
h<<MSI_LHXS<<12: ...      X_XXXX_XX
GI<<12           : ...      XXX

Total           : ..._0100_0X0X_XXXX_XX00_0XXX_0000_0000_0000

```

Block 2. Bits that can change in the MSI address

- i_is_delegated** is always 1 in M-domain.
- read_seq** requires only 2 bits.
- The bits from 3 to 9, and bits from 11 to 31 in **sourcecfg`** is read-only zero.
- The bit 11, bits from 15 to 17, and from 25 to 31 of **target** is read-only zero.
- The bits from 25 to 31 of **genmsi** is read-only zero.
- The bit 0 and 1 of addresses of registers in control regions is always 0.

5.5.3. Functional Coverage

Results.

NUM_SOURCE	Test cycles	Test cases	Result
128	5,000,000	104	Measurement was not possible due to a memory shortage.
4	5,000,000	104	99.1% (575/580)
2	20,000,000	104	100% (580/580)

Cover properties

When the following events occur, APLIC changes its internal state:

- Change in interrupt signal (0→1, 1→0)
- Write request
- MSI transmission

This coverage tests verified whether at least two of the above events occur simultaneously during testing. Examples of such patterns include:

- An interrupt with ID = 1 is inserted simultaneously with a write request to the **sourcecfg** with ID = 1.

- An interrupt with ID = 1 is inserted simultaneously with the sending of an MSI with ID = 1.
- An MSI with ID = 1 is sent simultaneously with a write request to the **sourcecfg** with ID = 1.

Tested both the transition from 0→1 and 1→0 for every interrupt source.

Regarding write requests, coverage was obtained for all writable registers included in the control region. Specifically, the following registers were evaluated. For **sourcecfg** and **target**, all registers except for ID = 0 are evaluated.

- **domaincfg**
- **sourcecfg** * (NUM_SOURCE - 1)
- **setip(num)**
- **(in_)clrip(num)**
- **setie(num)**
- **clrie(num)**
- **genmsi**
- **target** * (NUM_SOURCE - 1)

These tests verified all possible combinations of the current value of **D** and **SM** of **sourcecfg** and their value after writing. For example:

- Changing from D = 0, SM = LEVEL0 to D = 0, SM = LEVEL1.
- Changing from D = 1, CI = 0 to D = 0, SM = DETACHED

When a **setip(num)** write request occurs, APLIC behaves differently depending on the value of **SM** and **D** of **sourcecfg** for the corresponding ID.

- Write 6 to **setipnum** when **D** and **SM** in **sourcecfg[6]** are 0 and LEVEL1 respectively.

Verified each pattern of **SM** and **D** in **sourcecfg** when writes to **target** occurs. (Writes to **target** are only possible for the interrupt ID active in that domain.)

Waiver of Functional Coverage

- No MSI will be sent when **domaincfg.IE** is set to 0.
- No MSI will be sent when **sourcecfg.D** is set to one or **sourcecfg.SM** is set to INACTIVE or DETACHED.
- **sourcecfg.D** is always set to zero in S-domain.

5.6. Formal Verification

This section lists assertions for each module.

5.6.1. Gateway

Property	Input signals	Verification
If i_irq_source[i] is asserted and sourcecfg[i-1] is set to either EDGE1, EDGE0, LEVEL1, or LEVEL0 in the next cycle, then in the following cycle, o_set_pending_bit[i-1] is set to 1, and o_clr_sending_bit[i-1] is set to 0. Additionally, o_irq_rect is set to 1.	i_sourcecfg, i_irq_src	o_set_pending_gt, o_clr_sending_bit[i-1], o_irq_rect
If i_irq_source[i] is deasserted and sourcecfg[i-1] is set to either LEVEL1 or LEVEL0, then in the following cycle, o_set_pending_bit[i-1] is set to 0 and o_clr_sending_bit[i-1] is set to 1. Additionally, o_irq_rect is set to 0.	i_sourcecfg, i_irq_src	o_set_pending_gt, o_clr_sending_bit[i-1], o_irq_rect
If i_irq_source[i] is deasserted and sourcecfg[i-1] is set to either EDGE1 or EDGE0, then in the following cycle, o_set_pending_bit[i-1] is set to 0. Additionally, o_irq_rect is set to 0.	i_sourcecfg, i_irq_src	o_set_pending_gt, o_irq_rect
If i_irq_source[i] is asserted and sourcecfg[i-1] is set to either INACTIVE or DETACHED, then in the following cycle, o_set_pending_bit[i-1] is set to 0. Additionally, o_irq_rect is set to 0.	i_sourcecfg, i_irq_src	o_set_pending_gt, o_irq_rect

5.6.2. Register Control

Property	Input signals	Verification
When i_sourcecfg[i] is set to either DETACHED, EDGE1, EDGE0, LEVEL1, or LEVEL0, is_src_ready_for_setip[i] is set to 1. (For LEVEL1 and LEVEL0, this applies when i_irq_rect is 1.)	i_sourcecfg, (i_irq_rect)	is_src_ready_for_setip[i]
When i_sourcecfg[i] is set to either D = 1 or D = 0 and INACTIVE, is_src_inactive is set to 1.	i_sourcecfg	is_src_inactive
When is_src_ready_for_setip is 1 and a write occurs to setip(num(_le)) , a pending bits update request (set_pending_setip(num(_le))) is set to 1.	i_setip(num(_le)), src_ready_for_setip	set_pending_setip(num(_le))
When any of is_src_inactive[i] , i_clr_pending_gt[i] , clr_pending_in_clrip[i] , clr_pending_clripnum[i] , or i_irq_forwarded_tn[i] is set, o_irq_pending[i] is cleared to 0 in the next cycle.	is_src_inactive[i], i_clr_pending_gt[i], clr_pending_in_clrip[i], clr_pending_clripnum[i], i_irq_forwarded_tn[i]	o_irq_pending[i]
When set_pending_setip(num(_le))[i] is set to 1 and there is no pending bit clear request, o_irq_pending[i] is set to 1 in the next cycle.	is_src_ready_for_setip[i], is_src_inactive[i], i_clr_pending_gt[i], clr_pending_in_clrip[i], clr_pending_clripnum[i], i_irq_forwarded_tn[i]	o_irq_pending[i]
When i_set_pending_gt[i] is set to 1 and there is no pending bit clear request, o_irq_pending[i] is set to 1 in the next cycle.	i_set_pending_gt[i]	o_irq_pending[i]
When i_clr_pending_gt[i] is set to 1, o_irq_pending[i] is cleared to 0 in the next cycle.	i_clr_pending_gt[i]	o_irq_pending[i]
When a write occurs to setie(num) , set_enable_setie(num) is set.	i_setie(num)	set_enable_setie(num)
When set_pending_setie(num))[i] is set to 1 and there is no enable bit clear request, o_irq_enable[i] is set to 1 in the next cycle.	set_pending_setie(num))[i]	o_irq_enable[i]
When any of is_src_inactive[i] , clr_enable_clrie[i] , or clr_enable_clrienum[i] is set, o_irq_enable[i] is cleared to 0 in the next cycle.	is_src_inactive[i], clr_enable_clrie[i], clr_enable_clrienum[i]	o_irq_enable[i]
When i_is_delegated[i] is not set, sourcecfg[i] is cleared and locked in the next cycle.	i_is_delegated[i]	o_sourcecfg[i]
The value of o_irq_pending is set to o_setip .	i_irq_rect	o_setip
The value of i_irq_rect is set to o_in_clrip .	i_irq_rect	o_in_clrip
When is_src_inactive[i] is set to 1, o_irq_pending[i] and o_irq_enable[i] are cleared in the next cycle.	is_src_inactive[i]	o_irq_pending[i], o_irq_enable[i]

5.6.3. Request Handler

Property	Input signals	Verification
When <code>write_seq</code> is IDLE, if <code>i_ws_wreq.awvalid</code> and <code>i_ws_wreq.wvalid</code> become 1 at the same time, <code>write_seq</code> will transition through the states START, WAIT0, WAIT1, WRITE, RESPOND, FINISH, and IDLE every cycle.	<code>i_ws_wreq.awvalid</code> , <code>i_ws_wreq.wvalid</code> , <code>write_seq</code>	<code>write_seq</code>
When <code>write_seq</code> is WAIT1 or WRITE, if <code>is_in_domain_w[domain_id]</code> , <code>offset_valid_w[domain_id]</code> , (<code>cap_wstrb == 4'b1111</code>), and <code>is_cap_wdata_legal[domain_id]</code> are true, then <code>domain_wen[domain_id]</code> , <code>o_hc_psel[domain_id]</code> and <code>o_hc_pwrite[i]</code> are set to 1	<code>write_seq</code> , <code>is_in_domain_w[domain_id]</code> , <code>offset_valid_w[domain_id]</code> , <code>cap_wstrb</code> , <code>is_cap_wdata_legal[domain_id]</code>	<code>domain_wen[domain_id]</code> , <code>o_hc_psel[domain_id]</code> , <code>o_hc_pwrite[i]</code>
When <code>o_hc_psel[domain_id]</code> is set from 0 to 1, in the next cycle <code>o_hc_penable[domain_id]</code> is set to 1 and <code>o_hc_psel[domain_id]</code> is kept at 1, and in the cycle after that, both are set to 0.	<code>o_hc_psel[domain_id]</code>	<code>o_hc_psel[domain_id]</code> , <code>o_hc_penable[domain_id]</code>
When <code>read_seq</code> is START or READ, if <code>is_in_domain_r[domain_id]</code> and <code>offset_valid_r[domain_id]</code> are true, then <code>domain_ren[domain_id]</code> and <code>o_hc_psel[domain_id]</code> are set to 1, and then <code>o_hc_pwrite[i]</code> is set to 0.	<code>read_seq</code> , <code>is_in_domain_r[domain_id]</code> , <code>offset_valid_r[domain_id]</code>	<code>domain_ren[domain_id]</code> , <code>o_hc_psel[domain_id]</code> , <code>o_hc_pwrite[i]</code>
If <code>o_hc_penable[domain_id]</code> , <code>i_hc_pready[domain_id]</code> and <code>!o_hc_pwrite[domain_id]</code> are true, then <code>o_rs_resp.rvalid</code> will be 1 in the next cycle.	<code>o_hc_penable[domain_id]</code> , <code>i_hc_pready[domain_id]</code> , <code>!o_hc_pwrite[domain_id]</code>	<code>o_rs_resp.rvalid</code>

5.6.4. Target Notifier

Property	Input signals	Verification
When a write to <code>genmsi</code> occurs, <code>o_genmsi_busy</code> is set to 1 after 6 cycles. Immediately, <code>genmsi</code> is locked, and in the next cycle, <code>o_genmsi.Busy</code> is set to 1. In the cycle after that, <code>o_msi_valid</code> is set to 1, and the value calculated from <code>genmsi</code> is set to <code>o_msi_data</code> and <code>o_msi_addr</code> .	-	<code>o_genmsi</code>
When <code>i_domaincfg.IE</code> is 0, <code>o_msi_valid</code> will not become 1 if <code>o_genmsi_busy</code> is 0.	<code>i_domaincfg.IE</code> , <code>o_genmsi_busy</code>	<code>o_msi_valid</code>
If there is an <code>i</code> such that <code>i_irq_pending[i]</code> and <code>i_irq_enable[i]</code> are true, then <code>new_irq_detected</code> will be set to 1.	<code>i_irq_pending[i]</code> , <code>i_irq_enable[i]</code>	<code>new_irq_detected</code>
When <code>is_write_seq_busy</code> is 0, <code>o_msi_valid</code> becomes 1 if <code>o_genmsi_busy</code> is 1 or <code>new_irq_detected</code> and <code>i_domaincfg.IE</code> are both 1.	<code>o_genmsi_busy</code> , <code>new_irq_detected</code> , <code>i_domaincfg</code> , <code>is_write_seq_busy</code>	<code>o_msi_valid</code>
If <code>i_write_seq</code> is not IDLE or FINISH, <code>is_write_seq_busy</code> is set to 1.	<code>i_write_seq</code>	<code>is_write_seq_busy</code>
When <code>o_msi_valid</code> and <code>i_msi_ready</code> are set to 1, and <code>o_genmsi_busy</code> is 2'b0, the values of <code>o_msi_addr</code> and <code>o_msi_data</code> match the values calculated from <code>i_target</code> . In addition, <code>o_irq_forwarded_tn</code> is set to 1.	<code>o_msi_valid</code> , <code>i_msi_ready</code> , <code>o_genmsi_busy</code> , <code>i_target</code>	<code>o_msi_addr</code> , <code>o_msi_data</code> , <code>o_irq_forwarded_tn</code>

5.6.5. AXI5 Lite Read Subordinate

Property	Input signals	Verification
If <code>i_axi_arvalid</code> is 1 and <code>read_state</code> is IDLE, <code>o_axi_arready</code> becomes 1, and in the next cycle, <code>read_state</code> becomes READ. <code>i_axi_araddr</code> is assigned to <code>o_rs_rreq.araddr</code> . <code>i_axi_rid</code> is also assigned to <code>o_axi_rid</code> . Then, <code>o_rs_rreq.arvalid</code> becomes 1.	<code>i_axi_arvalid</code> , <code>read_state</code> , <code>i_axi_araddr</code> , <code>i_axi_rid</code>	<code>o_axi_arready</code> , <code>read_state</code> , <code>o_rs_rreq.araddr</code> , <code>o_axi_rid</code> , <code>o_rs_rreq.arvalid</code>
If <code>i_axi_rready</code> is 1 and <code>read_state</code> is RESPOND, <code>o_axi_rvalid</code> is immediately set to 1. In the next cycle, <code>read_state</code> becomes IDLE.	<code>i_axi_rready</code> , <code>read_state</code>	<code>read_state</code>
When <code>i_rs_resp.rvalid</code> is 1, in the next cycle, <code>i_rs_resp.rdata</code> is assigned to <code>o_axi_rdata</code> and <code>i_rs_resp.rresp</code> is assigned to <code>o_axi_rresp</code> . In addition, <code>read_state</code> becomes RESPOND.	<code>i_rs_resp.rvalid</code> , <code>i_rs_resp.rdata</code> , <code>i_rs_resp.rresp</code>	<code>o_axi_rdata</code> , <code>o_axi_rresp</code> , <code>read_state</code>

5.6.6. AXI5 Lite Write Manager

Property	Input signals	Verification
When <code>write_state</code> is IDLE, if <code>i_msi_valid[next_domain_id]</code> , then <code>o_msi_ready[next_domain_id]</code> becomes 1. In addition, in the next cycle, <code>write_state</code> becomes WRITE, and <code>o_axi_addr</code> and <code>o_axi_wdata</code> become equal to <code>i_msi_addr[next_domain_id]</code> and <code>i_msi_data[next_domain_id]</code> respectively.	<code>write_state</code> , <code>i_msi_valid[next_domain_id]</code> , <code>i_msi_addr[next_domain_id]</code> , <code>i_msi_data[next_domain_id]</code>	<code>write_state</code> , <code>o_msi_ready[next_domain_id]</code> , <code>o_axi_addr</code> , <code>o_axi_wdata</code>
If at least one bit of <code>i_genmsi_busy</code> is 1, then <code>next_domain_id</code> will be equal to that index.	<code>i_genmsi_busy</code>	<code>next_domain_id</code>
If <code>i_axi_awready</code> and <code>o_axi_awvalid</code> are true, <code>is_sent_awaddr</code> will be set to 1 in the next cycle.	<code>i_axi_awready</code> , <code>o_axi_awvalid</code>	<code>is_sent_awaddr</code>
If <code>i_axi_wready</code> and <code>o_axi_wvalid</code> are true, <code>is_sent_waddr</code> will be set to 1 in the next cycle.	<code>i_axi_wready</code> , <code>o_axi_wvalid</code>	<code>is_sent_waddr</code>
<code>o_axi_awprot</code> is always set to 0.	-	<code>o_axi_awprot</code>
<code>o_axi_bready</code> is always set to 0.	-	<code>o_axi_bready</code>
If both <code>is_sent_awaddr</code> and <code>is_sent_wdata</code> are 1, then <code>write_state</code> will be set to RESPOND.	<code>is_sent_awaddr</code> , <code>is_sent_wdata</code>	<code>write_state</code>
If <code>next_domain_id</code> is not equal to <code>i</code> , <code>o_msi_ready[i]</code> will be 0.	<code>next_domain_id</code>	<code>o_msi_ready[i]</code>

5.6.7. AXI5 Lite Write Subordinate

Property	Input signals	Verification
If <code>write_state</code> is IDLE and both <code>i_axi_awvalid</code> and <code>i_axi_wvalid</code> are 1, then <code>o_axi_awready</code> and <code>o_axi_wready</code> will immediately become 1, and in the next cycle <code>write_state</code> will become WRITE.	<code>write_state</code> , <code>i_axi_awvalid</code> , <code>i_axi_wvalid</code>	<code>write_state</code> , <code>o_axi_awready</code> , <code>o_axi_wready</code>
If <code>write_state</code> is WRITE, then <code>o_ws_wreq.awvalid</code> and <code>o_ws_wreq.wvalid</code> will be set to 1. <code>o_ws_wreq.awaddr</code> and <code>o_ws_wreq.wdata</code> will be assigned <code>i_axi_awaddr</code> and <code>i_axi_wdata</code> respectively.	<code>write_state</code> , <code>i_axi_awaddr</code> , <code>i_axi_wdata</code>	<code>o_ws_wreq.awvalid</code> , <code>o_ws_wreq.wvalid</code> , <code>o_ws_wreq.awaddr</code> , <code>o_ws_wreq.wdata</code>
If <code>i_axi_awvalid</code> and <code>o_axi_awready</code> are set to 1, then in the next cycle, <code>i_axi_awid</code> and <code>i_axi_awaddr</code> will be assigned to <code>o_axi_bid</code> and <code>o_ws_wreq.awaddr</code> respectively.	<code>i_axi_awvalid</code> , <code>o_axi_awready</code> , <code>i_axi_awid</code> , <code>i_axi_awaddr</code>	<code>o_axi_bid</code> , <code>o_ws_wreq.awaddr</code>
If <code>i_axi_wvalid</code> and <code>o_axi_wready</code> are true, then in the next cycle, <code>i_axi_wstrb</code> and <code>i_axi_wdata</code> will be assigned to <code>o_ws_wreq.wstrb</code> and <code>o_ws_wreq.wdata</code> respectively.	<code>i_axi_wvalid</code> , <code>o_axi_wready</code> , <code>i_axi_wstrb</code> , <code>i_axi_wdata</code>	<code>o_ws_wreq.wstrb</code> , <code>o_ws_wreq.wdata</code>
When <code>write_state</code> is RESPOND, <code>o_axi_bvalid</code> is set to 1 immediately. If <code>i_axi_bready</code> is 1, <code>write_state</code> is set to IDLE in the next cycle.	<code>write_state</code> , <code>i_axi_bready</code>	<code>write_state</code>
When <code>write_state</code> is WRITE and <code>i_ws_resp.bvalid</code> is 1, <code>write_state</code> will become RESPOND in the next cycle.	<code>write_state</code> , <code>i_ws_resp.bvalid</code>	<code>write_state</code>
If both <code>o_ws_wreq.wvalid</code> and <code>o_ws_wreq.awvalid</code> are 1, <code>i_ws_resp.bresp</code> is assigned to <code>o_axi_bresp</code> .	<code>o_ws_wreq.wvalid</code> , <code>o_ws_wreq.awvalid</code> , <code>i_ws_resp.bresp</code>	<code>o_axi_bresp</code>

Chapter 6. Appendix

6.1. Integration Guide

This TT-APLIC module requires external synchronizers for interrupt and reset signals. Other AXI signals like write strobe (o_msi_m0_wstrb) may also be necessary. Please see the integration example (tt_aplic) here: [aus-gitlab.local.tenstorrent.com/soc/aplic/-/blob/main/rtl/tt_aplic.sv](https://gitlab.local.tenstorrent.com/soc/aplic/-/blob/main/rtl/tt_aplic.sv)

```

module tt_aplic
#(

    parameter int unsigned NUM_SOURCE      = 128,
    parameter int unsigned NUM_HART        = 128,
    parameter int unsigned CFG_ID_WIDTH    = 10 ,

    parameter logic [55:0] CFG_ADDR_BASE_M = 56'h00_0000_1080_0000,
    parameter logic [55:0] CFG_ADDR_MASK_M = 56'h0F_FFFF_F00F_C000,
    parameter logic [55:0] CFG_ADDR_BASE_S = 56'h00_0000_2080_0000,
    parameter logic [55:0] CFG_ADDR_MASK_S = 56'h0F_FFFF_F00F_C000,
    parameter logic [55:0] MSI_BASE_ADDR_M = 56'h00_0000_4000_0000,
    parameter logic [55:0] MSI_BASE_ADDR_S = 56'h00_0000_4400_0000
) (

    // Global
    input          i_clk,
    input          i_reset_n,
    input          [NUM_SOURCE-1:1] i_irq_source,    // Interrupt source 0 is not available

    // Input Config Interface (AXI4-Lite - no USER)
    input          i_cfg_s0_awvalid,
    output         o_cfg_s0_awready,
    input          [CFG_ID_WIDTH-1:0] i_cfg_s0_awid,
    input          [55:0] i_cfg_s0_awaddr,
    input          [2:0] i_cfg_s0_awprot,
    input          i_cfg_s0_wvalid,
    output         o_cfg_s0_wready,
    input          [31:0] i_cfg_s0_wdata,
    input          [3:0] i_cfg_s0_wstrb,
    output         o_cfg_s0_bvalid,
    input          i_cfg_s0_bready,
    output         [CFG_ID_WIDTH-1:0] o_cfg_s0_bid,
    output         [1:0] o_cfg_s0_bresp,
    input          i_cfg_s0_arvalid,
    output         o_cfg_s0_arready,
    input          [CFG_ID_WIDTH-1:0] i_cfg_s0_arid,
    input          [55:0] i_cfg_s0_araddr,
    input          [2:0] i_cfg_s0_arprot, // Not used.
    output         o_cfg_s0_rvalid,
    input          i_cfg_s0_rready,
    output         [CFG_ID_WIDTH-1:0] o_cfg_s0_rid,
    output         [31:0] o_cfg_s0_rdata,
    output         [1:0] o_cfg_s0_rresp,
    output         o_msi_m0_awvalid,
    input          i_msi_m0_awready,
    output         [55:0] o_msi_m0_awaddr,
    output         [2:0] o_msi_m0_awprot,
    output         o_msi_m0_wvalid,
    input          i_msi_m0_wready,
    output         [3:0] o_msi_m0_wstrb, // always '1.
    output         [31:0] o_msi_m0_wdata,
    input          i_msi_m0_bvalid,
    output         o_msi_m0_bready,
    input          [1:0] i_msi_m0_bresp,
    output         o_msi_m0_arvalid,
    input          i_msi_m0_arready,
    output         [55:0] o_msi_m0_araddr ,
    output         [2:0] o_msi_m0_arprot ,

```

```

input                i_msi_m0_rvalid ,
output               o_msi_m0_rready ,
input                [31:0] i_msi_m0_rdata ,
input                [1:0] i_msi_m0_rresp ,

// DFT
input                i_test_mode,
input                i_scan_rst_n

);

/////////////////////////////////////////////////////////////////
// Submodule Instantiations
/////////////////////////////////////////////////////////////////

// Synchronize Interrupts
logic [NUM_SOURCE-1:1] irq_src;
logic reset_n;

// Reset Synchronization
tt_sync_reset #(
    .WIDTH ( 3 )           // number of sync flops
) tt_sync_reset_inst (
    .clk      ( i_clk      ), // input clock
    .rst_n    ( i_reset_n  ), // input reset
    .test_mode ( i_test_mode ), // test mode input
    .scan_rst_n ( i_scan_rst_n ), // scan reset input
    .sync_rst_n ( reset_n   ) // synchronized reset output, active low
);

// irq source synchronization
tt_sync3r #(
    .WIDTH (NUM_SOURCE - 1)
) tt_sync3r_irq_source_inst (
    .i_clk      ( i_clk      ),
    .i_d        ( i_irq_source ),
    .i_reset_n  ( reset_n    ),
    .o_q        ( irq_src    )
);

/////////////////////////////////////////////////////////////////
// APLIC TOP
/////////////////////////////////////////////////////////////////

aplic_top #(
    .NUM_SOURCE      (NUM_SOURCE      ),
    .NUM_HART        (NUM_HART        ),
    .CFG_AWID_WIDTH  (CFG_ID_WIDTH    ),
    .CFG_ARID_WIDTH  (CFG_ID_WIDTH    ),
    .CFG_ADDR_BASE_M(CFG_ADDR_BASE_M),
    .CFG_ADDR_MASK_M(CFG_ADDR_MASK_M),
    .CFG_ADDR_BASE_S(CFG_ADDR_BASE_S),
    .CFG_ADDR_MASK_S(CFG_ADDR_MASK_S),
    .MSI_BASE_ADDR_M(MSI_BASE_ADDR_M),
    .MSI_BASE_ADDR_S(MSI_BASE_ADDR_S)
) aplic_top_inst (
    .i_clk      (i_clk      ),
    .i_reset_n  (reset_n),
    .i_irq_source ({irq_src, 1'b0}),

    .i_cfg_s0_awvalid (i_cfg_s0_awvalid),
    .o_cfg_s0_awready (o_cfg_s0_awready),
    .i_cfg_s0_awid    (i_cfg_s0_awid    ),
    .i_cfg_s0_awaddr   (i_cfg_s0_awaddr   ),

    .i_cfg_s0_wvalid   (i_cfg_s0_wvalid   ),
    .o_cfg_s0_wready   (o_cfg_s0_wready   ),
    .i_cfg_s0_wdata    (i_cfg_s0_wdata    ),
    .i_cfg_s0_wstrb    (i_cfg_s0_wstrb    ),

```

```

.o_cfg_s0_bvalid  (o_cfg_s0_bvalid ),
.i_cfg_s0_bready  (i_cfg_s0_bready ),
.o_cfg_s0_bid     (o_cfg_s0_bid     ),
.o_cfg_s0_bresp   (o_cfg_s0_bresp   ),

.i_cfg_s0_arvalid (i_cfg_s0_arvalid),
.o_cfg_s0_arready (o_cfg_s0_arready),
.i_cfg_s0_arid    (i_cfg_s0_arid    ),
.i_cfg_s0_araddr  (i_cfg_s0_araddr  ),

.o_cfg_s0_rvalid  (o_cfg_s0_rvalid ),
.i_cfg_s0_rready  (i_cfg_s0_rready ),
.o_cfg_s0_rid     (o_cfg_s0_rid     ),
.o_cfg_s0_rdata   (o_cfg_s0_rdata   ),
.o_cfg_s0_rresp   (o_cfg_s0_rresp   ),

.o_msi_m0_awvalid (o_msi_m0_awvalid),
.i_msi_m0_awready (i_msi_m0_awready),
.o_msi_m0_awaddr  (o_msi_m0_awaddr ),
.o_msi_m0_awprot  (o_msi_m0_awprot ),

.o_msi_m0_wvalid  (o_msi_m0_wvalid ),
.i_msi_m0_wready  (i_msi_m0_wready ),
.o_msi_m0_wdata   (o_msi_m0_wdata   ),

.i_msi_m0_bvalid  (i_msi_m0_bvalid ),
.o_msi_m0_bready  (o_msi_m0_bready ),
.i_msi_m0_bresp   (i_msi_m0_bresp   ),

.o_msi_m0_arvalid (o_msi_m0_arvalid), //o_msi_m0_arvalid is always 0
.i_msi_m0_arready (i_msi_m0_arready), //i_msi_m0_arready is unused
.o_msi_m0_araddr  (o_msi_m0_araddr ), //o_msi_m0_araddr is always 0
.o_msi_m0_arprot  (o_msi_m0_arprot ), //o_msi_m0_arprot is always 0

.i_msi_m0_rvalid  (i_msi_m0_rvalid ), //i_msi_m0_rvalid is unused
.o_msi_m0_rready  (o_msi_m0_rready ), //o_msi_m0_rready is always 1
.i_msi_m0_rdata   (i_msi_m0_rdata ), //i_msi_m0_rdata[31:0] is unused
.i_msi_m0_rresp   (i_msi_m0_rresp )  //i_msi_m0_rresp[1:0] is unused
);

assign o_msi_m0_wstrb = '1;

endmodule

```

Block 3. tt_aplic

6.2. Programming Guide

6.2.1. TT-APLIC Control Region Initialization

To initialize the TT-APLIC control region:

1. Initialize the control region of the M-domain.
 - a. domaincfg register
 - b. sourcecfg register
 - c. other registers
2. Initialize the control region of the S-domain.
 - a. domaincfg register
 - b. sourcecfg register
 - c. other registers
3. Begin actions such as asserting a wired interrupt, writing to GENMSI, or writing to SETIP.

For example, you can initialize the registers using the following procedure, assuming that

1. The maximum hart ID is 127 (NUM_HART=128)
2. There are 127 total interrupt signals (ID = 1,2,...,127. ID 0 is not available.)
3. There are two domains (M-domain and S-domain)
4. The RISC-V CPU attached to the TT-APLIC supports 5 guest interrupt files.

Step 1: initialize the M-domain config register in software.

1. Set 1 to IE (**domaincfg[8]**) in the M-domain **domaincfg**. DM and BE are read-only in the TT-APLIC implementation.
2. Initialize **sourcecfg[n]** ($n=1,2,...,127$) in the M-domain to the desired value.
 - a. If the interrupt n is delegated to the S-domain, then set D=1 (**sourcecfg[n][10]**) and CI=0 (**sourcecfg[n][9:0]**).
 - b. If the interrupt n is taken by the M-domain, then set D=0 (**sourcecfg[n][10]**). Program SM (**sourcecfg[n][2:0]**) depending on the reaction from IRQ signal (e.g., ignore, making MSI from rising edge, etc...)
3. When the interrupt n is programmed to be taken by the M-domain, initialize (**target[n]** ($n=1,2,...,127$)) in the M-domain to the desired value.
 - a. EIID (**target[n][10:0]**) to be MSI data. This is exposed in [10:0] of the MSI write data.
 - b. Set the hart index (**target[n][31:18]**) to the target hart ID.
 - c. The hart index should be a value between 0 and 127. The TT-APLIC ignores write requests with other values.
4. Set **setie[n/32][n%32]** ($n=1,2,...,127$) to 1 in the M-domain to start accepting an interrupt from the corresponding signal.
5. If required, program **setip[n/32][n%32]** or write n to **setipnum** or **setipnum_le** in the M-domain to explicitly send a MSI.

Step 2: initialize the S-domain config register in software.

1. Set 1 to IE (**domaincfg[8]**) in the S-domain **domaincfg**. DM and BE are read-only in this TT-APLIC implementation.
2. Initialize **sourcecfg** in the S-domain to the desired value if the interrupt is delegated to the S-domain.
 - a. Unlike the M-domain, the D-bit (**sourcecfg[n][10]**) cannot be programmed to 1 because the S-domain is the lowest level.
3. Initialize **target** in the S-domain. HI [31:18] and EIID [10:0] are treated as being the same as the M-domain. **target[n][17:12]** is treated as Guest Hart ID.
 - a. The Guest index should be a value between 0 and 5. The TT-APLIC ignores write requests with other values.
4. Set 1 to **setie[n/32][n%32]** ($n=1,2,...,127$) in the S-domain to start accepting an interrupt from a corresponding signal.
5. If required, program **setip[n/32][n%32]** or write n to **setipnum** or **setipnum_le** in the S-domain to explicitly send a MSI.

After completing initialization, software may start generating MSIs by asserting incoming interrupt signals, writing appropriate values for **genmsi**, **setip**, **setipnum(_le)**

All settings, such as the pending bit, enable bit, and target, for inactive interrupt sources are maintained as read-only zeros. Any writes to configuration registers associated with inactive sources will be ignored. Registers other than domaincfg and sourcecfg in each domain must therefore only be initialized after initializing the corresponding number of sourcecfg and activating the interrupt source.

An inactive interrupt is defined as either of the following:

- The D field in the corresponding sourcecfg is set to 1.
- The D field in the corresponding sourcecfg is set to 0 and the SM field is set to INACTIVE (000b).

The sourcecfg of the S-domain must only be initialized after initializing the sourcecfg for the same interrupt number in the M-domain. When the D field is set to 1 and the CI field is set to 0 in the M-domain, the interrupt source is delegated from the M-domain to the S-domain. In this case, a value of 0 in the CI field indicates the domain number of the S-domain.

The domaincfg should be initialized before other registers but can be set at any time. The IE bit is the only writable bit in the domaincfg and acts as a switch for toggling MSI transmission on and off. It neither affects nor is affected by the other register's settings. However, setting the domaincfg late will delay MSI transmissions until it is configured. This delay could lead to an unexpected interrupt processing order or even result in losing the interrupt request.

Note: Toggling the IE bit only affects MSI transmissions triggered by wired interrupt sources but does not impact MSI transmission triggered by GENMSI. In other words, MSI transmission via GENMSI can be forwarded even when IE is set to 0. Please set the corresponding configuration registers (sourcecfg, target) appropriately before accepting interruptions via the wired sources or writing setup.

6.2.2. Additional information.

The write request data should be 32 bit.

Write operations to registers should use 32-bit data. The WSTRB should always be 4'b1111. Prepare 32-bit data and send the write request even when only specific bits need to be written. In this case, for fields that should not be updated, you must first read the register value and copy it to the data to be written.

Always use legal values when writing to a register. If you write an illegal value, then the write to that field will be ignored and the previous value will be retained. If there are both legal and illegal fields in the same write request, then only the legal fields will be written, and the illegal fields will be ignored with the previous values retained. This operation will not cause the TT-APLIC to lose consistency but may cause unexpected behavior.

Legal values

A legal value refers to the following values:

- sourcecfg
 - D = 1 and CI = 0
 - D = 0 and SM = INACTIVE (000b), DETACHED (001b), EDGE1 (100b), EDGE0 (101b), LEVEL1 (110b), or LEVEL0 (111b)
- genmsi
 - $0 \leq HI < \text{NUM_HART}$
- target
 - $0 \leq HI < \text{NUM_HART}$
 - $0 \leq GI < \text{NUM_GEILEN}$

The values for any other fields are recognized as legal values.

Checking results of write requests

Verify that the expected value has been written by sending a read request to the register. An "Okay" response to a write request does not mean that the expected value was written. For example, if you attempt to write legal values to some fields of a target register and illegal values to others, then only the legal values will be written with the illegal fields ignored and an "Okay" write response.

Additionally, Attempting to write a value to the target register of an inactive interrupt number will be ignored, and the target register will remain at 0. The response to the write request in this case will also be "Okay".

Write requests after initialization

You can make post-initialization write requests to these registers and change the TT-APLIC settings. Use caution when making changes to sourcecfg, such as changing SM from LEVEL1 to LEVEL0 and the wire is kept at 0, which risks sending an unexpected MSI. You can prevent this by setting either the:

- clrie register to set the relevant enable bit to 0
- domaincfg IE to 0 to stop the transmission of MSI.

System reset

Resetting the system initializes each register to its default value. Please see the next section for the default register values.

6.3. aplic_control_region_m address map

- Absolute Address: 0x0
- Base Offset: 0x0
- Size: 0x4000

For each interrupt domain that an APLIC supports, there is a dedicated memory-mapped control region for managing interrupts in that domain. This control region is for the M-level domain. This control region size is 16 KiB and populated by a set of 32-bit registers.

Offset	Identifier	Name
0x0000	domaincfg	domaincfg
0x0004	sourcecfg[0]	sourcecfg
0x0008	sourcecfg[1]	sourcecfg
0x000C	sourcecfg[2]	sourcecfg
0x0010	sourcecfg[3]	sourcecfg
0x0014	sourcecfg[4]	sourcecfg
0x0018	sourcecfg[5]	sourcecfg
0x001C	sourcecfg[6]	sourcecfg
0x0020	sourcecfg[7]	sourcecfg
0x0024	sourcecfg[8]	sourcecfg
0x0028	sourcecfg[9]	sourcecfg
0x002C	sourcecfg[10]	sourcecfg
0x0030	sourcecfg[11]	sourcecfg
0x0034	sourcecfg[12]	sourcecfg
0x0038	sourcecfg[13]	sourcecfg
0x003C	sourcecfg[14]	sourcecfg
0x0040	sourcecfg[15]	sourcecfg
0x0044	sourcecfg[16]	sourcecfg
0x0048	sourcecfg[17]	sourcecfg
0x004C	sourcecfg[18]	sourcecfg
0x0050	sourcecfg[19]	sourcecfg
0x0054	sourcecfg[20]	sourcecfg
0x0058	sourcecfg[21]	sourcecfg
0x005C	sourcecfg[22]	sourcecfg
0x0060	sourcecfg[23]	sourcecfg
0x0064	sourcecfg[24]	sourcecfg
0x0068	sourcecfg[25]	sourcecfg
0x006C	sourcecfg[26]	sourcecfg
0x0070	sourcecfg[27]	sourcecfg
0x0074	sourcecfg[28]	sourcecfg
0x0078	sourcecfg[29]	sourcecfg
0x007C	sourcecfg[30]	sourcecfg
0x0080	sourcecfg[31]	sourcecfg
0x0084	sourcecfg[32]	sourcecfg
0x0088	sourcecfg[33]	sourcecfg
0x008C	sourcecfg[34]	sourcecfg
0x0090	sourcecfg[35]	sourcecfg
0x0094	sourcecfg[36]	sourcecfg
0x0098	sourcecfg[37]	sourcecfg
0x009C	sourcecfg[38]	sourcecfg

Offset	Identifier	Name
0x00A0	sourcecfg[39]	sourcecfg
0x00A4	sourcecfg[40]	sourcecfg
0x00A8	sourcecfg[41]	sourcecfg
0x00AC	sourcecfg[42]	sourcecfg
0x00B0	sourcecfg[43]	sourcecfg
0x00B4	sourcecfg[44]	sourcecfg
0x00B8	sourcecfg[45]	sourcecfg
0x00BC	sourcecfg[46]	sourcecfg
0x00C0	sourcecfg[47]	sourcecfg
0x00C4	sourcecfg[48]	sourcecfg
0x00C8	sourcecfg[49]	sourcecfg
0x00CC	sourcecfg[50]	sourcecfg
0x00D0	sourcecfg[51]	sourcecfg
0x00D4	sourcecfg[52]	sourcecfg
0x00D8	sourcecfg[53]	sourcecfg
0x00DC	sourcecfg[54]	sourcecfg
0x00E0	sourcecfg[55]	sourcecfg
0x00E4	sourcecfg[56]	sourcecfg
0x00E8	sourcecfg[57]	sourcecfg
0x00EC	sourcecfg[58]	sourcecfg
0x00F0	sourcecfg[59]	sourcecfg
0x00F4	sourcecfg[60]	sourcecfg
0x00F8	sourcecfg[61]	sourcecfg
0x00FC	sourcecfg[62]	sourcecfg
0x0100	sourcecfg[63]	sourcecfg
0x0104	sourcecfg[64]	sourcecfg
0x0108	sourcecfg[65]	sourcecfg
0x010C	sourcecfg[66]	sourcecfg
0x0110	sourcecfg[67]	sourcecfg
0x0114	sourcecfg[68]	sourcecfg
0x0118	sourcecfg[69]	sourcecfg
0x011C	sourcecfg[70]	sourcecfg
0x0120	sourcecfg[71]	sourcecfg
0x0124	sourcecfg[72]	sourcecfg
0x0128	sourcecfg[73]	sourcecfg
0x012C	sourcecfg[74]	sourcecfg
0x0130	sourcecfg[75]	sourcecfg
0x0134	sourcecfg[76]	sourcecfg
0x0138	sourcecfg[77]	sourcecfg
0x013C	sourcecfg[78]	sourcecfg
0x0140	sourcecfg[79]	sourcecfg
0x0144	sourcecfg[80]	sourcecfg
0x0148	sourcecfg[81]	sourcecfg
0x014C	sourcecfg[82]	sourcecfg
0x0150	sourcecfg[83]	sourcecfg
0x0154	sourcecfg[84]	sourcecfg
0x0158	sourcecfg[85]	sourcecfg
0x015C	sourcecfg[86]	sourcecfg
0x0160	sourcecfg[87]	sourcecfg

Offset	Identifier	Name
0x0164	sourcecfg[88]	sourcecfg
0x0168	sourcecfg[89]	sourcecfg
0x016C	sourcecfg[90]	sourcecfg
0x0170	sourcecfg[91]	sourcecfg
0x0174	sourcecfg[92]	sourcecfg
0x0178	sourcecfg[93]	sourcecfg
0x017C	sourcecfg[94]	sourcecfg
0x0180	sourcecfg[95]	sourcecfg
0x0184	sourcecfg[96]	sourcecfg
0x0188	sourcecfg[97]	sourcecfg
0x018C	sourcecfg[98]	sourcecfg
0x0190	sourcecfg[99]	sourcecfg
0x0194	sourcecfg[100]	sourcecfg
0x0198	sourcecfg[101]	sourcecfg
0x019C	sourcecfg[102]	sourcecfg
0x01A0	sourcecfg[103]	sourcecfg
0x01A4	sourcecfg[104]	sourcecfg
0x01A8	sourcecfg[105]	sourcecfg
0x01AC	sourcecfg[106]	sourcecfg
0x01B0	sourcecfg[107]	sourcecfg
0x01B4	sourcecfg[108]	sourcecfg
0x01B8	sourcecfg[109]	sourcecfg
0x01BC	sourcecfg[110]	sourcecfg
0x01C0	sourcecfg[111]	sourcecfg
0x01C4	sourcecfg[112]	sourcecfg
0x01C8	sourcecfg[113]	sourcecfg
0x01CC	sourcecfg[114]	sourcecfg
0x01D0	sourcecfg[115]	sourcecfg
0x01D4	sourcecfg[116]	sourcecfg
0x01D8	sourcecfg[117]	sourcecfg
0x01DC	sourcecfg[118]	sourcecfg
0x01E0	sourcecfg[119]	sourcecfg
0x01E4	sourcecfg[120]	sourcecfg
0x01E8	sourcecfg[121]	sourcecfg
0x01EC	sourcecfg[122]	sourcecfg
0x01F0	sourcecfg[123]	sourcecfg
0x01F4	sourcecfg[124]	sourcecfg
0x01F8	sourcecfg[125]	sourcecfg
0x01FC	sourcecfg[126]	sourcecfg
0x1BC0	mmsiaddrcfg	mmsiaddrcfg
0x1BC4	mmsiaddrcfggh	mmsiaddrcfggh
0x1BC8	smsiaddrcfg	smsiaddrcfg
0x1BCC	smsiaddrcfggh	smsiaddrcfggh
0x1C00	setip[0]	setip
0x1C04	setip[1]	setip
0x1C08	setip[2]	setip
0x1C0C	setip[3]	setip
0x1CDC	setipnum	setipnum
0x1D00	in_clrip[0]	in_clrip

Offset	Identifier	Name
0x1D04	in_clrip[1]	in_clrip
0x1D08	in_clrip[2]	in_clrip
0x1DOC	in_clrip[3]	in_clrip
0x1DDC	clripnum	clripnum
0x1E00	setie[0]	setie
0x1E04	setie[1]	setie
0x1E08	setie[2]	setie
0x1E0C	setie[3]	setie
0x1EDC	setienum	setienum
0x1F00	clrie[0]	clrie
0x1F04	clrie[1]	clrie
0x1F08	clrie[2]	clrie
0x1F0C	clrie[3]	clrie
0x1FDC	clrienum	clrienum
0x2000	setipnum_le	setipnum_le
0x2004	setipnum_be	setipnum_be
0x3000	genmsi	genmsi
0x3004	target[0]	target
0x3008	target[1]	target
0x300C	target[2]	target
0x3010	target[3]	target
0x3014	target[4]	target
0x3018	target[5]	target
0x301C	target[6]	target
0x3020	target[7]	target
0x3024	target[8]	target
0x3028	target[9]	target
0x302C	target[10]	target
0x3030	target[11]	target
0x3034	target[12]	target
0x3038	target[13]	target
0x303C	target[14]	target
0x3040	target[15]	target
0x3044	target[16]	target
0x3048	target[17]	target
0x304C	target[18]	target
0x3050	target[19]	target
0x3054	target[20]	target
0x3058	target[21]	target
0x305C	target[22]	target
0x3060	target[23]	target
0x3064	target[24]	target
0x3068	target[25]	target
0x306C	target[26]	target
0x3070	target[27]	target
0x3074	target[28]	target
0x3078	target[29]	target
0x307C	target[30]	target
0x3080	target[31]	target

Offset	Identifier	Name
0x3084	target[32]	target
0x3088	target[33]	target
0x308C	target[34]	target
0x3090	target[35]	target
0x3094	target[36]	target
0x3098	target[37]	target
0x309C	target[38]	target
0x30A0	target[39]	target
0x30A4	target[40]	target
0x30A8	target[41]	target
0x30AC	target[42]	target
0x30B0	target[43]	target
0x30B4	target[44]	target
0x30B8	target[45]	target
0x30BC	target[46]	target
0x30C0	target[47]	target
0x30C4	target[48]	target
0x30C8	target[49]	target
0x30CC	target[50]	target
0x30D0	target[51]	target
0x30D4	target[52]	target
0x30D8	target[53]	target
0x30DC	target[54]	target
0x30E0	target[55]	target
0x30E4	target[56]	target
0x30E8	target[57]	target
0x30EC	target[58]	target
0x30F0	target[59]	target
0x30F4	target[60]	target
0x30F8	target[61]	target
0x30FC	target[62]	target
0x3100	target[63]	target
0x3104	target[64]	target
0x3108	target[65]	target
0x310C	target[66]	target
0x3110	target[67]	target
0x3114	target[68]	target
0x3118	target[69]	target
0x311C	target[70]	target
0x3120	target[71]	target
0x3124	target[72]	target
0x3128	target[73]	target
0x312C	target[74]	target
0x3130	target[75]	target
0x3134	target[76]	target
0x3138	target[77]	target
0x313C	target[78]	target
0x3140	target[79]	target
0x3144	target[80]	target

Offset	Identifier	Name
0x3148	target[81]	target
0x314C	target[82]	target
0x3150	target[83]	target
0x3154	target[84]	target
0x3158	target[85]	target
0x315C	target[86]	target
0x3160	target[87]	target
0x3164	target[88]	target
0x3168	target[89]	target
0x316C	target[90]	target
0x3170	target[91]	target
0x3174	target[92]	target
0x3178	target[93]	target
0x317C	target[94]	target
0x3180	target[95]	target
0x3184	target[96]	target
0x3188	target[97]	target
0x318C	target[98]	target
0x3190	target[99]	target
0x3194	target[100]	target
0x3198	target[101]	target
0x319C	target[102]	target
0x31A0	target[103]	target
0x31A4	target[104]	target
0x31A8	target[105]	target
0x31AC	target[106]	target
0x31B0	target[107]	target
0x31B4	target[108]	target
0x31B8	target[109]	target
0x31BC	target[110]	target
0x31C0	target[111]	target
0x31C4	target[112]	target
0x31C8	target[113]	target
0x31CC	target[114]	target
0x31D0	target[115]	target
0x31D4	target[116]	target
0x31D8	target[117]	target
0x31DC	target[118]	target
0x31E0	target[119]	target
0x31E4	target[120]	target
0x31E8	target[121]	target
0x31EC	target[122]	target
0x31F0	target[123]	target
0x31F4	target[124]	target
0x31F8	target[125]	target
0x31FC	target[126]	target

6.3.1. domaincfg register

- Absolute Address: 0x0
- Base Offset: 0x0
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0	BE	r	0x0	Bit BE (Big-Endian)
2	DM	r	0x1	Field DM (Delivery Mode)
7	RSVDZ1	r	0x0	-
8	IE	rw	0x0	Bit IE (Interrupt Enable)
31:24	RSVDZO	r	0x80	-

BE field

This bit is read-only 0. This APLIC supports little-endian only. This is a WARL field that determines the byte order for most registers in the interrupt domain's memory-mapped control region. If BE = 0, byte order is little-endian, and if BE = 1, it is big-endian.

DM field

This bit is read-only one because This APLIC supports MSI mode only. This determines how this interrupt domain delivers interrupts to harts. The two possible values for DM are: 0 = direct delivery mode 1 = MSI delivery mode

RSVDZ1 field

This is read-only 0.

IE field

This is a global enable for all active interrupt sources at this interrupt domain. Only when IE = 1 are pending-and-enabled interrupts actually signaled or forwarded to harts. At system reset, this is initialized to zero.

RSVDZO field

This is read-only 0x80.

6.3.2. sourcecfg register

- Absolute Address: 0x4
- Base Offset: 0x4
- Size: 0x4
- Array Dimensions: [127]
- Array Stride: 0x4
- Total Size: 0x1FC

For each possible interrupt source *i*, register sourcecfg controls the source mode for source *i* in this interrupt domain as well as any delegation of the source to a child domain. When source *i* is not implemented, or appears in this domain not to be implemented, sourcecfg is read-only zero. If source *i* was not delegated to this domain and is then changed (at the parent domain) to become delegated to this domain, sourcecfg remains zero until successfully written with a nonzero value

Bits	Identifier	Access	Reset	Name
2:0	CI_L_SM	rw	0x0	Child Index L and SM (Source Mode)
9:3	CI_H	rw	0x0	Child Index
10	D	rw	0x0	D (Delegate)

CI_L_SM field

This field is used as Child Index L or SM. The SM (Source Mode) field is WARL and controls whether the interrupt source is active in this domain, and if so, what values or transitions on the incoming wire are interpreted as interrupts. The values allowed for SM and their meanings are listed below. Inactive (zero) is always supported for field SM. Implementations are free to choose, independently for each interrupt source, what other values are supported for SM. The encoding of the SM (Source Mode) field of a sourcecfg register when bit D = 0 is as follows:

Value	Name	Description
0	Inactive	Inactive in this domain (and not delegated)
1	Detached	Active, detached from the source wire
2-3	-	Reserved
4	Edge1	Active, edge-sensitive; interrupt asserted on rising edge
5	Edge0	Active, edge-sensitive; interrupt asserted on falling edge
6	Level1	Active, level-sensitive; interrupt asserted on when high
7	Level0	Active, level-sensitive; interrupt asserted on when low

When the domain is not delegated, this should be read-only zero and the update is ignored.

CI_H field

Child Index is a WLRL field that specifies the interrupt domain to which this source is delegated. For an interrupt domain with C child domains, this field must be able to hold integer values in the range 0 to C - 1. Each interrupt domain has a fixed mapping from these index numbers to child domains. If an interrupt domain has no children in the domain hierarchy, bit D cannot be set to one in any sourcecfg register for that domain. For such a leaf domain, attempting to write a sourcecfg register with a value that has bit 10 = 1 causes the entire register to be set to zero instead. The field is used in conjunction with CI_L. When the domain is not delegated, this should be read-only zero and the update is ignored.

D field

Bit 10 of sourcecfg is a 1-bit field called D (Delegate). If D = 1, source i is delegated to a child domain, and if D = 0, it is not delegated to a child domain. Interpretation of the rest of sourcecfg depends on field D. When the domain is not delegated, this should be read-only zero and the update is ignored.

6.3.3. mmsiaddrcfg register

- Absolute Address: 0x1BC0
- Base Offset: 0x1BC0
- Size: 0x4

This is read-only zero for this system.

Bits	Identifier	Access	Reset	Name
31:0	LBPPN	r	0x0	Low Base PPN

LBPPN field

This is read-only zero for this system.

6.3.4. mmsiaddrcfg register

- Absolute Address: 0x1BC4
- Base Offset: 0x1BC4
- Size: 0x4

This is read-only 0x8000_0000 (for M-domain) or 0x0000_0000 (for S-domain) for this system.

Bits	Identifier	Access	Reset	Name
11:0	HBPPN	r	0x0	High_Base_PPN
15:12	LHXW	r	0x0	LHXW
18:16	HHXW	r	0x0	HHXW
22:20	LHXS	r	0x0	LHXS
28:24	HHXS	r	0x0	HHXS
31	L	r	0x1	L

HBPPN field

This is read-only zero for this system.

LHXW field

This is read-only zero for this system.

HHXW field

This is read-only zero for this system.

LHXS field

This is read-only zero for this system.

HHXS field

This is read-only zero for this system.

L field

This is read-only one for this system..

6.3.5. smsiaddrcfg register

- Absolute Address: 0x1BC8
- Base Offset: 0x1BC8
- Size: 0x4

This is read-only zero for this system.

Bits	Identifier	Access	Reset	Name
31:0	LBPPN	r	0x0	Low_Base_PPN

LBPPN field

This is read-only zero for this system.

6.3.6. smsiaddrcfg register

- Absolute Address: 0x1BCC
- Base Offset: 0x1BCC
- Size: 0x4

This is read-only zero for this system.

Bits	Identifier	Access	Reset	Name
11:0	HBPPN	r	0x0	High Base PPN
22:20	LHXS	r	0x0	LHXS

HBPPN field

This is read-only zero for this system.

LHXS field

This is read-only zero for this system.

6.3.7. setip register

- Absolute Address: 0x1C00
- Base Offset: 0x1C00
- Size: 0x4
- Array Dimensions: [4]
- Array Stride: 0x4
- Total Size: 0x10

Reading or writing register setip[k] reads or potentially modifies the pending bits for interrupt sources $k \times 32$ through $k \times 32 + 31$. For an implemented interrupt source i within that range, the pending bit for source i corresponds with register bit $(i \bmod 32)$. A read of a setip register returns the pending bits of the corresponding interrupt sources. Bit positions in the result value that do not correspond to an implemented interrupt source (such as bit 0 of setip[0]) are zeros. On a write to a setip register, for each bit that is one in the 32-bit value written, if that bit position corresponds to an active interrupt source, the interrupt-pending bit for that source is set to one if possible.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	FULL REG
31:0	WRDATA	w	0x0	FULL REG

RDDATA field

APLIC copies the data of the corresponding pending bits to this register.

WRDATA field

APLIC copies the contents of this field to the pending bit by woset, then clears the entire field.

6.3.8. setipnum register

- Absolute Address: 0x1CDC
- Base Offset: 0x1CDC
- Size: 0x4

If i is an active interrupt source number in the domain, writing 32-bit value i to register setipnum causes the pending bit for source i to be set to one if possible. See Section 4.7 for exactly when a pending bit may be set by writing to setipnum. A write to setipnum is ignored if the value written is not an active interrupt source number in the domain. A read of setipnum always returns zero.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

A read of this register always returns zero.

WRDATA field

Ignore writes if the value is not an active interrupt source number.

6.3.9. in_clrip register

- Absolute Address: 0x1D00
- Base Offset: 0x1D00
- Size: 0x4
- Array Dimensions: [4]
- Array Stride: 0x4
- Total Size: 0x10

Reading register `in_clrip[k]` returns the rectified input values for interrupt sources $k \times 32$ through $k \times 32 + 31$, while writing in `clrip[k]` potentially modifies the pending bits for the same sources. For an implemented interrupt source i within the specified range, source i corresponds with register bit $(i \bmod 32)$. A read of an `in_clrip` register returns the rectified input values of the corresponding interrupt sources. Bit positions in the result value that do not correspond to an implemented interrupt source (such as bit 0 of `in_clrip[0]`) are zeros. On a write to an `in_clrip` register, for each bit that is one in the 32-bit value written, if that bit position corresponds to an active interrupt source, the interrupt-pending bit for that source is cleared if possible.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

Reading register `in_clrip[k]` returns the rectified input values for interrupt sources $k \times 32$ through $k \times 32 + 31$.

WRDATA field

Writing in `in_clrip[k]` potentially modifies the pending bits for the same sources.

6.3.10. clripnum register

- Absolute Address: 0x1DDC
- Base Offset: 0x1DDC
- Size: 0x4

If i is an active interrupt source number in the domain, writing 32-bit value i to register `clripnum` causes the pending bit for source i to be cleared if possible. A write to `clripnum` is ignored if the value written is not an active interrupt source number in the domain. A read of `clripnum` always returns zero.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

A read of this register always returns zero.

WRDATA field

writing 32-bit value i to register `clripnum` causes the pending bit for source i to be cleared if possible.

6.3.11. setie register

- Absolute Address: 0x1E00
- Base Offset: 0x1E00
- Size: 0x4
- Array Dimensions: [4]
- Array Stride: 0x4
- Total Size: 0x10

Reading or writing register `setie[k]` reads or potentially modifies the enable bits for interrupt sources $k \times 32$ through $k \times 32 + 31$. For an implemented interrupt source i within that range, the enable bit for source i corresponds with register bit $(i \bmod 32)$. A read of a `setie` register returns the enable bits of the corresponding interrupt sources. Bit positions in the result value that do not correspond to an implemented interrupt source (such as bit 0 of `setie[0]`) are zeros. On a write to a `setie` register, for each bit that is one in the 32-bit value written, if that bit position corresponds to an active interrupt source, the interrupt-enable bit for that source is set to one.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

A read of a `setie` register returns the enable bits of the corresponding interrupt sources.

WRDATA field

6.3.12. setienum register

- Absolute Address: 0x1EDC
- Base Offset: 0x1EDC
- Size: 0x4

If i is an active interrupt source number in the domain, writing 32-bit value i to register `setienum` causes the enable bit for source i to be set to one. A write to `setienum` is ignored if the value written is not an active interrupt source number in the domain. A read of `setienum` always returns zero.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

A read of this register always returns zero.

WRDATA field

A read of this register always returns zero.

6.3.13. clrie register

- Absolute Address: 0x1F00
- Base Offset: 0x1F00
- Size: 0x4
- Array Dimensions: [4]
- Array Stride: 0x4
- Total Size: 0x10

Writing register `clrie[k]` potentially modifies the enable bits for interrupt sources $k \times 32$ through $k \times 32 + 31$. For an implemented interrupt source i within that range, the enable bit for source i corresponds with register bit $(i \bmod 32)$. On a write to a `clrie` register, for each bit that is one in the 32-bit value written, the interrupt- enable bit for that source is cleared. A read of a `clrie` register always returns zero.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

A read of this register always returns zero.

WRDATA field

A read of this register always returns zero.

6.3.14. `clrienum` register

- Absolute Address: 0x1FDC
- Base Offset: 0x1FDC
- Size: 0x4

If i is an active interrupt source number in the domain, writing 32-bit value i to register `clrienum` causes the enable bit for source i to be cleared. A write to `clrienum` is ignored if the value written is not an active interrupt source number in the domain. A read of `clrienum` always returns zero.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

A read of this register always returns zero.

WRDATA field

6.3.15. `setipnum_le` register

- Absolute Address: 0x2000
- Base Offset: 0x2000
- Size: 0x4

This has the same function as `setipnum` in this system. Register `setipnum_le` acts identically to `setipnum` except that byte order is always little-endian, as though field BE (Big-Endian) of register `domaincfg` is zero.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

A read of this register always returns zero.

WRDATA field

Ignore writes if the value is not an active interrupt source number.

6.3.16. setipnum_be register

- Absolute Address: 0x2004
- Base Offset: 0x2004
- Size: 0x4

This is read-only zero for this system. For systems that are little-endian-only, with domaincfg.BE hardwired to zero, setipnum_be need not be implemented, in which case the four bytes at this offset are simply read-only zeros like other reserved bytes.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA

RDDATA field

This is read-only zero.

6.3.17. genmsi register

- Absolute Address: 0x3000
- Base Offset: 0x3000
- Size: 0x4

When the interrupt domain is configured in MSI delivery mode (domaincfg.DM = 1), register genmsi can be used to cause an extempore MSI to be sent from the APLIC to a hart. The main purpose for this function is to assist in establishing a temporary known ordering between a hart's writes to the APLIC's registers and the transmission of MSIs from the APLIC to the hart. Extempore MSIs are not affected by the IE bit of the domain's domaincfg register. An extempore MSI is sent even if domaincfg.IE = 0. When the interrupt domain is configured in direct delivery mode (domaincfg.DM = 0), register genmsi is read-only zero.

Bits	Identifier	Access	Reset	Name
10:0	EIID	rw	0x0	EIID
12	Busy	r	0x0	Busy
31:18	HI	rw	0x0	Hart Index

EIID field

This is a WARL field.

Busy field

This is a read-only field. The Busy bit is ordinarily zero (false), but a write to genmsi causes Busy to become one (true), indicating an extempore MSI is pending. The Hart Index field specifies the destination hart, and EIID (External Interrupt Identity) specifies the data value for the MSI. Fields Hart Index and EIID have the same formats and behavior as in a target register. For a machine-level interrupt domain, an extempore MSI is sent to the destination hart at machine level, and for a supervisor-level interrupt domain, an extempore MSI is sent to the destination hart at supervisor level. A pending extempore MSI should be sent by the APLIC with minimal delay. Once it has left the APLIC and the APLIC is able to accept a new write to genmsi for another extempore MSI, Busy reverts to false. All MSIs previously sent from this APLIC to the same hart must be visible at the hart's IMSIC before the extempore MSI becomes visible at the hart's IMSIC. While Busy is true, writes to genmsi are ignored.

HI field

This is a WLRL field.

6.3.18. target register

- Absolute Address: 0x3004
- Base Offset: 0x3004
- Size: 0x4
- Array Dimensions: [127]

- Array Stride: 0x4
- Total Size: 0x1FC

If interrupt source i is inactive in this domain, register target is read-only zero. If source i is active, target determines the hart to which interrupts from the source are signaled or forwarded. The exact interpretation of target depends on the delivery mode configured by field DM of register domaincfg. DM is read-only one in this APLIC.

Bits	Identifier	Access	Reset	Name
10:0	EIID	rw	0x0	EIID
17:12	GI	rw	0x0	Guest Index
31:18	HI	rw	0x0	Hart Index

EIID field

The remaining field EIID (External Interrupt Identity) specifies the data value for those MSIs, eventually becoming the minor identity for an external interrupt at the target hart.

GI field

If the interrupt domain is at supervisor level and the domain's harts implement the RISC-V Privileged Architecture's hypervisor extension, then Guest Index is a WLRL field that must be able to hold all integer values in the range 0 through GEILEN. (Parameter GEILEN is defined by the Privileged Architecture's hypervisor extension.) Otherwise, field Guest Index is read-only zero. For a supervisor-level interrupt domain, a nonzero Guest Index is the number of the target hart's guest interrupt file to which MSIs will be sent. When Guest Index is zero, MSIs from a supervisor-level domain are forwarded to the target hart at supervisor level. For a machine-level domain, Guest Index is read-only zero, and MSIs are forwarded to a target hart always at machine level. Together, fields Hart Index and Guest Index of register target determine the address for MSIs forwarded for interrupt source i .

HI field

Together, fields Hart Index and Guest Index of register target determine the address for MSIs forwarded for interrupt source i .

6.4. aplc_control_region_s address map

- Absolute Address: 0x0
- Base Offset: 0x0
- Size: 0x4000

For each interrupt domain that an APLIC supports, there is a dedicated memory-mapped control region for managing interrupts in that domain. This control region is for the S domain. This control region size is 16 KiB and populated by a set of 32-bit registers.

Offset	Identifier	Name
0x0000	domaincfg	domaincfg
0x0004	sourcecfg[0]	sourcecfg
0x0008	sourcecfg[1]	sourcecfg
0x000C	sourcecfg[2]	sourcecfg
0x0010	sourcecfg[3]	sourcecfg
0x0014	sourcecfg[4]	sourcecfg
0x0018	sourcecfg[5]	sourcecfg
0x001C	sourcecfg[6]	sourcecfg
0x0020	sourcecfg[7]	sourcecfg
0x0024	sourcecfg[8]	sourcecfg
0x0028	sourcecfg[9]	sourcecfg
0x002C	sourcecfg[10]	sourcecfg
0x0030	sourcecfg[11]	sourcecfg
0x0034	sourcecfg[12]	sourcecfg
0x0038	sourcecfg[13]	sourcecfg
0x003C	sourcecfg[14]	sourcecfg
0x0040	sourcecfg[15]	sourcecfg
0x0044	sourcecfg[16]	sourcecfg
0x0048	sourcecfg[17]	sourcecfg
0x004C	sourcecfg[18]	sourcecfg
0x0050	sourcecfg[19]	sourcecfg
0x0054	sourcecfg[20]	sourcecfg
0x0058	sourcecfg[21]	sourcecfg
0x005C	sourcecfg[22]	sourcecfg
0x0060	sourcecfg[23]	sourcecfg
0x0064	sourcecfg[24]	sourcecfg
0x0068	sourcecfg[25]	sourcecfg
0x006C	sourcecfg[26]	sourcecfg
0x0070	sourcecfg[27]	sourcecfg
0x0074	sourcecfg[28]	sourcecfg
0x0078	sourcecfg[29]	sourcecfg
0x007C	sourcecfg[30]	sourcecfg
0x0080	sourcecfg[31]	sourcecfg
0x0084	sourcecfg[32]	sourcecfg
0x0088	sourcecfg[33]	sourcecfg
0x008C	sourcecfg[34]	sourcecfg
0x0090	sourcecfg[35]	sourcecfg
0x0094	sourcecfg[36]	sourcecfg
0x0098	sourcecfg[37]	sourcecfg
0x009C	sourcecfg[38]	sourcecfg
0x00A0	sourcecfg[39]	sourcecfg

Offset	Identifier	Name
0x00A4	sourcecfg[40]	sourcecfg
0x00A8	sourcecfg[41]	sourcecfg
0x00AC	sourcecfg[42]	sourcecfg
0x00B0	sourcecfg[43]	sourcecfg
0x00B4	sourcecfg[44]	sourcecfg
0x00B8	sourcecfg[45]	sourcecfg
0x00BC	sourcecfg[46]	sourcecfg
0x00C0	sourcecfg[47]	sourcecfg
0x00C4	sourcecfg[48]	sourcecfg
0x00C8	sourcecfg[49]	sourcecfg
0x00CC	sourcecfg[50]	sourcecfg
0x00D0	sourcecfg[51]	sourcecfg
0x00D4	sourcecfg[52]	sourcecfg
0x00D8	sourcecfg[53]	sourcecfg
0x00DC	sourcecfg[54]	sourcecfg
0x00E0	sourcecfg[55]	sourcecfg
0x00E4	sourcecfg[56]	sourcecfg
0x00E8	sourcecfg[57]	sourcecfg
0x00EC	sourcecfg[58]	sourcecfg
0x00F0	sourcecfg[59]	sourcecfg
0x00F4	sourcecfg[60]	sourcecfg
0x00F8	sourcecfg[61]	sourcecfg
0x00FC	sourcecfg[62]	sourcecfg
0x0100	sourcecfg[63]	sourcecfg
0x0104	sourcecfg[64]	sourcecfg
0x0108	sourcecfg[65]	sourcecfg
0x010C	sourcecfg[66]	sourcecfg
0x0110	sourcecfg[67]	sourcecfg
0x0114	sourcecfg[68]	sourcecfg
0x0118	sourcecfg[69]	sourcecfg
0x011C	sourcecfg[70]	sourcecfg
0x0120	sourcecfg[71]	sourcecfg
0x0124	sourcecfg[72]	sourcecfg
0x0128	sourcecfg[73]	sourcecfg
0x012C	sourcecfg[74]	sourcecfg
0x0130	sourcecfg[75]	sourcecfg
0x0134	sourcecfg[76]	sourcecfg
0x0138	sourcecfg[77]	sourcecfg
0x013C	sourcecfg[78]	sourcecfg
0x0140	sourcecfg[79]	sourcecfg
0x0144	sourcecfg[80]	sourcecfg
0x0148	sourcecfg[81]	sourcecfg
0x014C	sourcecfg[82]	sourcecfg
0x0150	sourcecfg[83]	sourcecfg
0x0154	sourcecfg[84]	sourcecfg
0x0158	sourcecfg[85]	sourcecfg
0x015C	sourcecfg[86]	sourcecfg
0x0160	sourcecfg[87]	sourcecfg
0x0164	sourcecfg[88]	sourcecfg

Offset	Identifier	Name
0x0168	sourcecfg[89]	sourcecfg
0x016C	sourcecfg[90]	sourcecfg
0x0170	sourcecfg[91]	sourcecfg
0x0174	sourcecfg[92]	sourcecfg
0x0178	sourcecfg[93]	sourcecfg
0x017C	sourcecfg[94]	sourcecfg
0x0180	sourcecfg[95]	sourcecfg
0x0184	sourcecfg[96]	sourcecfg
0x0188	sourcecfg[97]	sourcecfg
0x018C	sourcecfg[98]	sourcecfg
0x0190	sourcecfg[99]	sourcecfg
0x0194	sourcecfg[100]	sourcecfg
0x0198	sourcecfg[101]	sourcecfg
0x019C	sourcecfg[102]	sourcecfg
0x01A0	sourcecfg[103]	sourcecfg
0x01A4	sourcecfg[104]	sourcecfg
0x01A8	sourcecfg[105]	sourcecfg
0x01AC	sourcecfg[106]	sourcecfg
0x01B0	sourcecfg[107]	sourcecfg
0x01B4	sourcecfg[108]	sourcecfg
0x01B8	sourcecfg[109]	sourcecfg
0x01BC	sourcecfg[110]	sourcecfg
0x01C0	sourcecfg[111]	sourcecfg
0x01C4	sourcecfg[112]	sourcecfg
0x01C8	sourcecfg[113]	sourcecfg
0x01CC	sourcecfg[114]	sourcecfg
0x01D0	sourcecfg[115]	sourcecfg
0x01D4	sourcecfg[116]	sourcecfg
0x01D8	sourcecfg[117]	sourcecfg
0x01DC	sourcecfg[118]	sourcecfg
0x01E0	sourcecfg[119]	sourcecfg
0x01E4	sourcecfg[120]	sourcecfg
0x01E8	sourcecfg[121]	sourcecfg
0x01EC	sourcecfg[122]	sourcecfg
0x01F0	sourcecfg[123]	sourcecfg
0x01F4	sourcecfg[124]	sourcecfg
0x01F8	sourcecfg[125]	sourcecfg
0x01FC	sourcecfg[126]	sourcecfg
0x1BC0	mmsiaddrcfg	mmsiaddrcfg
0x1BC4	mmsiaddrcfggh	mmsiaddrcfggh
0x1BC8	smsiaddrcfg	smsiaddrcfg
0x1BCC	smsiaddrcfggh	smsiaddrcfggh
0x1C00	setip[0]	setip
0x1C04	setip[1]	setip
0x1C08	setip[2]	setip
0x1C0C	setip[3]	setip
0x1CDC	setipnum	setipnum
0x1D00	in_clrip[0]	in_clrip
0x1D04	in_clrip[1]	in_clrip

Offset	Identifier	Name
0x1D08	in_clrip[2]	in_clrip
0x1D0C	in_clrip[3]	in_clrip
0x1DDC	clripnum	clripnum
0x1E00	setie[0]	setie
0x1E04	setie[1]	setie
0x1E08	setie[2]	setie
0x1EOC	setie[3]	setie
0x1EDC	setienum	setienum
0x1F00	clrie[0]	clrie
0x1F04	clrie[1]	clrie
0x1F08	clrie[2]	clrie
0x1FOC	clrie[3]	clrie
0x1FDC	clrienum	clrienum
0x2000	setipnum_le	setipnum_le
0x2004	setipnum_be	setipnum_be
0x3000	genmsi	genmsi
0x3004	target[0]	target
0x3008	target[1]	target
0x300C	target[2]	target
0x3010	target[3]	target
0x3014	target[4]	target
0x3018	target[5]	target
0x301C	target[6]	target
0x3020	target[7]	target
0x3024	target[8]	target
0x3028	target[9]	target
0x302C	target[10]	target
0x3030	target[11]	target
0x3034	target[12]	target
0x3038	target[13]	target
0x303C	target[14]	target
0x3040	target[15]	target
0x3044	target[16]	target
0x3048	target[17]	target
0x304C	target[18]	target
0x3050	target[19]	target
0x3054	target[20]	target
0x3058	target[21]	target
0x305C	target[22]	target
0x3060	target[23]	target
0x3064	target[24]	target
0x3068	target[25]	target
0x306C	target[26]	target
0x3070	target[27]	target
0x3074	target[28]	target
0x3078	target[29]	target
0x307C	target[30]	target
0x3080	target[31]	target
0x3084	target[32]	target

Offset	Identifier	Name
0x3088	target[33]	target
0x308C	target[34]	target
0x3090	target[35]	target
0x3094	target[36]	target
0x3098	target[37]	target
0x309C	target[38]	target
0x30A0	target[39]	target
0x30A4	target[40]	target
0x30A8	target[41]	target
0x30AC	target[42]	target
0x30B0	target[43]	target
0x30B4	target[44]	target
0x30B8	target[45]	target
0x30BC	target[46]	target
0x30C0	target[47]	target
0x30C4	target[48]	target
0x30C8	target[49]	target
0x30CC	target[50]	target
0x30D0	target[51]	target
0x30D4	target[52]	target
0x30D8	target[53]	target
0x30DC	target[54]	target
0x30E0	target[55]	target
0x30E4	target[56]	target
0x30E8	target[57]	target
0x30EC	target[58]	target
0x30F0	target[59]	target
0x30F4	target[60]	target
0x30F8	target[61]	target
0x30FC	target[62]	target
0x3100	target[63]	target
0x3104	target[64]	target
0x3108	target[65]	target
0x310C	target[66]	target
0x3110	target[67]	target
0x3114	target[68]	target
0x3118	target[69]	target
0x311C	target[70]	target
0x3120	target[71]	target
0x3124	target[72]	target
0x3128	target[73]	target
0x312C	target[74]	target
0x3130	target[75]	target
0x3134	target[76]	target
0x3138	target[77]	target
0x313C	target[78]	target
0x3140	target[79]	target
0x3144	target[80]	target
0x3148	target[81]	target

Offset	Identifier	Name
0x314C	target[82]	target
0x3150	target[83]	target
0x3154	target[84]	target
0x3158	target[85]	target
0x315C	target[86]	target
0x3160	target[87]	target
0x3164	target[88]	target
0x3168	target[89]	target
0x316C	target[90]	target
0x3170	target[91]	target
0x3174	target[92]	target
0x3178	target[93]	target
0x317C	target[94]	target
0x3180	target[95]	target
0x3184	target[96]	target
0x3188	target[97]	target
0x318C	target[98]	target
0x3190	target[99]	target
0x3194	target[100]	target
0x3198	target[101]	target
0x319C	target[102]	target
0x31A0	target[103]	target
0x31A4	target[104]	target
0x31A8	target[105]	target
0x31AC	target[106]	target
0x31B0	target[107]	target
0x31B4	target[108]	target
0x31B8	target[109]	target
0x31BC	target[110]	target
0x31C0	target[111]	target
0x31C4	target[112]	target
0x31C8	target[113]	target
0x31CC	target[114]	target
0x31D0	target[115]	target
0x31D4	target[116]	target
0x31D8	target[117]	target
0x31DC	target[118]	target
0x31E0	target[119]	target
0x31E4	target[120]	target
0x31E8	target[121]	target
0x31EC	target[122]	target
0x31F0	target[123]	target
0x31F4	target[124]	target
0x31F8	target[125]	target
0x31FC	target[126]	target

6.4.1. domaincfg register

- Absolute Address: 0x0

- Base Offset: 0x0
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0	BE	r	0x0	Bit BE (Big-Endian)
2	DM	r	0x1	Field DM (Delivery Mode)
7	RSVDZ1	r	0x0	-
8	IE	rw	0x0	Bit IE (Interrupt Enable)
31:24	RSVDZ0	r	0x80	-

BE field

This bit is read-only 0. This APLIC supports little-endian only. This is a WARL field that determines the byte order for most registers in the interrupt domain's memory-mapped control region. If BE = 0, byte order is little-endian, and if BE = 1, it is big-endian.

DM field

This bit is read-only one because This APLIC supports MSI mode only. This determines how this interrupt domain delivers interrupts to harts. The two possible values for DM are: 0 = direct delivery mode 1 = MSI delivery mode

RSVDZ1 field

This is read-only 0.

IE field

This is a global enable for all active interrupt sources at this interrupt domain. Only when IE = 1 are pending-and-enabled interrupts actually signaled or forwarded to harts. At system reset, this is initialized to zero.

RSVDZ0 field

This is read-only 0x80.

6.4.2. sourcecfg register

- Absolute Address: 0x4
- Base Offset: 0x4
- Size: 0x4
- Array Dimensions: [127]
- Array Stride: 0x4
- Total Size: 0x1FC

For each possible interrupt source *i*, register sourcecfg controls the source mode for source *i* in this interrupt domain as well as any delegation of the source to a child domain. When source *i* is not implemented, or appears in this domain not to be implemented, sourcecfg is read-only zero. If source *i* was not delegated to this domain and is then changed (at the parent domain) to become delegated to this domain, sourcecfg remains zero until successfully written with a nonzero value

Bits	Identifier	Access	Reset	Name
2:0	CI_L_SM	rw	0x0	Child Index L and SM (Source Mode)
9:3	CI_H	rw	0x0	Child Index
10	D	rw	0x0	D (Delegate)

CI_L_SM field

This field is used as Child Index L or SM. The SM (Source Mode) field is WARL and controls whether the interrupt source is active in this domain, and if so, what values or transitions on the incoming wire are interpreted as interrupts. The values allowed for SM

and their meanings are listed below. Inactive (zero) is always supported for field SM. Implementations are free to choose, independently for each interrupt source, what other values are supported for SM. The encoding of the SM (Source Mode) field of a sourcecfg register when bit D = 0 is as follows:

Value	Name	Description
0	Inactive	Inactive in this domain (and not delegated)
1	Detached	Active, detached from the source wire
2-3	-	Reserved
4	Edge1	Active, edge-sensitive; interrupt asserted on rising edge
5	Edge0	Active, edge-sensitive; interrupt asserted on falling edge
6	Level1	Active, level-sensitive; interrupt asserted on when high
7	Level0	Active, level-sensitive; interrupt asserted on when low

When the domain is not delegated, this should be read-only zero and the update is ignored.

CI_H field

Child Index is a WLRL field that specifies the interrupt domain to which this source is delegated. For an interrupt domain with C child domains, this field must be able to hold integer values in the range 0 to C - 1. Each interrupt domain has a fixed mapping from these index numbers to child domains. If an interrupt domain has no children in the domain hierarchy, bit D cannot be set to one in any sourcecfg register for that domain. For such a leaf domain, attempting to write a sourcecfg register with a value that has bit 10 = 1 causes the entire register to be set to zero instead. The field is used in conjunction with CI_L. When the domain is not delegated, this should be read-only zero and the update is ignored.

D field

Bit 10 of sourcecfg is a 1-bit field called D (Delegate). If D = 1, source i is delegated to a child domain, and if D = 0, it is not delegated to a child domain. Interpretation of the rest of sourcecfg depends on field D. When the domain is not delegated, this should be read-only zero and the update is ignored.

6.4.3. mmsiaddrcfg register

- Absolute Address: 0x1BC0
- Base Offset: 0x1BC0
- Size: 0x4

This is read-only zero for this system.

Bits	Identifier	Access	Reset	Name
31:0	LBPPN	r	0x0	Low Base PPN

LBPPN field

This is read-only zero for this system.

6.4.4. mmsiaddrcfg register

- Absolute Address: 0x1BC4
- Base Offset: 0x1BC4
- Size: 0x4

This is read-only 0x8000_0000 (for M-domain) or 0x0000_0000 (for S-domain) for this system.

Bits	Identifier	Access	Reset	Name
11:0	HBPPN	r	0x0	High_Base_PPN
15:12	LHXW	r	0x0	LHXW
18:16	HHXW	r	0x0	HHXW

Bits	Identifier	Access	Reset	Name
22:20	LHXS	r	0x0	LHXS
28:24	HHXS	r	0x0	HHXS
31	L	r	0x0	L

HBPPN field

This is read-only zero for this system.

LHXW field

This is read-only zero for this system.

HHXW field

This is read-only zero for this system.

LHXS field

This is read-only zero for this system.

HHXS field

This is read-only zero for this system.

L field

This is read-only one for this system..

6.4.5. smsiaddrcfg register

- Absolute Address: 0x1BC8
- Base Offset: 0x1BC8
- Size: 0x4

This is read-only zero for this system.

Bits	Identifier	Access	Reset	Name
31:0	LBPPN	r	0x0	Low_Base_PPN

LBPPN field

This is read-only zero for this system.

6.4.6. smsiaddrcfgh register

- Absolute Address: 0x1BCC
- Base Offset: 0x1BCC
- Size: 0x4

This is read-only zero for this system.

Bits	Identifier	Access	Reset	Name
11:0	HBPPN	r	0x0	High Base PPN
22:20	LHXS	r	0x0	LHXS

HBPPN field

This is read-only zero for this system.

LHXS field

This is read-only zero for this system.

6.4.7. setip register

- Absolute Address: 0x1C00
- Base Offset: 0x1C00
- Size: 0x4
- Array Dimensions: [4]
- Array Stride: 0x4
- Total Size: 0x10

Reading or writing register setip[k] reads or potentially modifies the pending bits for interrupt sources $k \times 32$ through $k \times 32 + 31$. For an implemented interrupt source i within that range, the pending bit for source i corresponds with register bit $(i \bmod 32)$. A read of a setip register returns the pending bits of the corresponding interrupt sources. Bit positions in the result value that do not correspond to an implemented interrupt source (such as bit 0 of setip[0]) are zeros. On a write to a setip register, for each bit that is one in the 32-bit value written, if that bit position corresponds to an active interrupt source, the interrupt-pending bit for that source is set to one if possible.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	FULL REG
31:0	WRDATA	w	0x0	FULL REG

RDDATA field

APLIC copies the data of the corresponding pending bits to this register.

WRDATA field

APLIC copies the contents of this field to the pending bit by woset, then clears the entire field.

6.4.8. setipnum register

- Absolute Address: 0x1CDC
- Base Offset: 0x1CDC
- Size: 0x4

If i is an active interrupt source number in the domain, writing 32-bit value i to register setipnum causes the pending bit for source i to be set to one if possible. See Section 4.7 for exactly when a pending bit may be set by writing to setipnum. A write to setipnum is ignored if the value written is not an active interrupt source number in the domain. A read of setipnum always returns zero.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

A read of this register always returns zero.

WRDATA field

Ignore writes if the value is not an active interrupt source number.

6.4.9. in_clrip register

- Absolute Address: 0x1D00
- Base Offset: 0x1D00
- Size: 0x4
- Array Dimensions: [4]
- Array Stride: 0x4
- Total Size: 0x10

Reading register `in_clrip[k]` returns the rectified input values for interrupt sources $k \times 32$ through $k \times 32 + 31$, while writing in `clrip[k]` potentially modifies the pending bits for the same sources. For an implemented interrupt source i within the specified range, source i corresponds with register bit $(i \bmod 32)$. A read of an in clrip register returns the rectified input values of the corresponding interrupt sources. Bit positions in the result value that do not correspond to an implemented interrupt source (such as bit 0 of in clrip[0]) are zeros. On a write to an in clrip register, for each bit that is one in the 32-bit value written, if that bit position corresponds to an active interrupt source, the interrupt-pending bit for that source is cleared if possible.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

Reading register `in_clrip[k]` returns the rectified input values for interrupt sources $k \times 32$ through $k \times 32 + 31$.

WRDATA field

Writing in `in_clrip[k]` potentially modifies the pending bits for the same sources.

6.4.10. clripnum register

- Absolute Address: 0x1DDC
- Base Offset: 0x1DDC
- Size: 0x4

If i is an active interrupt source number in the domain, writing 32-bit value i to register `clripnum` causes the pending bit for source i to be cleared if possible. A write to `clripnum` is ignored if the value written is not an active interrupt source number in the domain. A read of `clripnum` always returns zero.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

A read of this register always returns zero.

WRDATA field

writing 32-bit value i to register `clripnum` causes the pending bit for source i to be cleared if possible.

6.4.11. setie register

- Absolute Address: 0x1E00
- Base Offset: 0x1E00
- Size: 0x4
- Array Dimensions: [4]

- Array Stride: 0x4
- Total Size: 0x10

Reading or writing register `setie[k]` reads or potentially modifies the enable bits for interrupt sources $k \times 32$ through $k \times 32 + 31$. For an implemented interrupt source i within that range, the enable bit for source i corresponds with register bit $(i \bmod 32)$. A read of a `setie` register returns the enable bits of the corresponding interrupt sources. Bit positions in the result value that do not correspond to an implemented interrupt source (such as bit 0 of `setie[0]`) are zeros. On a write to a `setie` register, for each bit that is one in the 32-bit value written, if that bit position corresponds to an active interrupt source, the interrupt-enable bit for that source is set to one.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

A read of a `setie` register returns the enable bits of the corresponding interrupt sources.

WRDATA field

6.4.12. setienum register

- Absolute Address: 0x1EDC
- Base Offset: 0x1EDC
- Size: 0x4

If i is an active interrupt source number in the domain, writing 32-bit value i to register `setienum` causes the enable bit for source i to be set to one. A write to `setienum` is ignored if the value written is not an active interrupt source number in the domain. A read of `setienum` always returns zero.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

A read of this register always returns zero.

WRDATA field

A read of this register always returns zero.

6.4.13. clrie register

- Absolute Address: 0x1F00
- Base Offset: 0x1F00
- Size: 0x4
- Array Dimensions: [4]
- Array Stride: 0x4
- Total Size: 0x10

Writing register `clrie[k]` potentially modifies the enable bits for interrupt sources $k \times 32$ through $k \times 32 + 31$. For an implemented interrupt source i within that range, the enable bit for source i corresponds with register bit $(i \bmod 32)$. On a write to a `clrie` register, for each bit that is one in the 32-bit value written, the interrupt- enable bit for that source is cleared. A read of a `clrie` register always returns zero.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA

Bits	Identifier	Access	Reset	Name
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

A read of this register always returns zero.

WRDATA field

A read of this register always returns zero.

6.4.14. clrienum register

- Absolute Address: 0x1FDC
- Base Offset: 0x1FDC
- Size: 0x4

If *i* is an active interrupt source number in the domain, writing 32-bit value *i* to register clrienum causes the enable bit for source *i* to be cleared. A write to clrienum is ignored if the value written is not an active interrupt source number in the domain. A read of clrienum always returns zero.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

A read of this register always returns zero.

WRDATA field**6.4.15. setipnum_le register**

- Absolute Address: 0x2000
- Base Offset: 0x2000
- Size: 0x4

This has the same function as setipnum in this system. Register setipnum_le acts identically to setipnum except that byte order is always little-endian, as though field BE (Big-Endian) of register domaincfg is zero.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA
31:0	WRDATA	w	0x0	WRDATA

RDDATA field

A read of this register always returns zero.

WRDATA field

Ignore writes if the value is not an active interrupt source number.

6.4.16. setipnum_be register

- Absolute Address: 0x2004
- Base Offset: 0x2004

- Size: 0x4

This is read-only zero for this system. For systems that are little-endian-only, with domaincfg.BE hardwired to zero, setipnum_be need not be implemented, in which case the four bytes at this offset are simply read-only zeros like other reserved bytes.

Bits	Identifier	Access	Reset	Name
31:0	RDDATA	r	0x0	RDDATA

RDDATA field

This is read-only zero.

6.4.17. genmsi register

- Absolute Address: 0x3000
- Base Offset: 0x3000
- Size: 0x4

When the interrupt domain is configured in MSI delivery mode (domaincfg.DM = 1), register genmsi can be used to cause an extempore MSI to be sent from the APLIC to a hart. The main purpose for this function is to assist in establishing a temporary known ordering between a hart's writes to the APLIC's registers and the transmission of MSIs from the APLIC to the hart. Extempore MSIs are not affected by the IE bit of the domain's domaincfg register. An extempore MSI is sent even if domaincfg.IE = 0. When the interrupt domain is configured in direct delivery mode (domaincfg.DM = 0), register genmsi is read-only zero.

Bits	Identifier	Access	Reset	Name
10:0	EIID	rw	0x0	EIID
12	Busy	r	0x0	Busy
31:18	HI	rw	0x0	Hart Index

EIID field

This is a WARL field.

Busy field

This is a read-only field. The Busy bit is ordinarily zero (false), but a write to genmsi causes Busy to become one (true), indicating an extempore MSI is pending. The Hart Index field specifies the destination hart, and EIID (External Interrupt Identity) specifies the data value for the MSI. Fields Hart Index and EIID have the same formats and behavior as in a target register. For a machine-level interrupt domain, an extempore MSI is sent to the destination hart at machine level, and for a supervisor-level interrupt domain, an extempore MSI is sent to the destination hart at supervisor level. A pending extempore MSI should be sent by the APLIC with minimal delay. Once it has left the APLIC and the APLIC is able to accept a new write to genmsi for another extempore MSI, Busy reverts to false. All MSIs previously sent from this APLIC to the same hart must be visible at the hart's IMSIC before the extempore MSI becomes visible at the hart's IMSIC. While Busy is true, writes to genmsi are ignored.

HI field

This is a WLRL field.

6.4.18. target register

- Absolute Address: 0x3004
- Base Offset: 0x3004
- Size: 0x4
- Array Dimensions: [127]
- Array Stride: 0x4
- Total Size: 0x1FC

If interrupt source *i* is inactive in this domain, register target is read-only zero. If source *i* is active, target determines the hart to which interrupts from the source are signaled or forwarded. The exact interpretation of target depends on the delivery mode configured by field DM of register domaincfg. DM is read-only one in this APLIC.

Bits	Identifier	Access	Reset	Name
10:0	EIID	rw	0x0	EIID
17:12	GI	rw	0x0	Guest Index
31:18	HI	rw	0x0	Hart Index

EIID field

The remaining field EIID (External Interrupt Identity) specifies the data value for those MSIs, eventually becoming the minor identity for an external interrupt at the target hart.

GI field

If the interrupt domain is at supervisor level and the domain's harts implement the RISC-V Privileged Architecture's hypervisor extension, then Guest Index is a WRLR field that must be able to hold all integer values in the range 0 through GEILEN. (Parameter GEILEN is defined by the Privileged Architecture's hypervisor extension.) Otherwise, field Guest Index is read-only zero. For a supervisor-level interrupt domain, a nonzero Guest Index is the number of the target hart's guest interrupt file to which MSIs will be sent. When Guest Index is zero, MSIs from a supervisor-level domain are forwarded to the target hart at supervisor level. For a machine-level domain, Guest Index is read-only zero, and MSIs are forwarded to a target hart always at machine level. Together, fields Hart Index and Guest Index of register target determine the address for MSIs forwarded for interrupt source *i*.

HI field

Together, fields Hart Index and Guest Index of register target determine the address for MSIs forwarded for interrupt source *i*.

Chapter 7. Revision History

No	Version	Date	Author	Description of Changes
1	v0.7.0	09/28/2024	Shota Minato	First Release.
2	v0.9.0	01/31/2025	Shota Minato	Major revised version released.
3	v0.9.1	03/04/2025	Shota Minato	Fixed important notes and the legal value of the target. The TT-APLIC may ignore incoming interrupts if they are pulses shorter than six cycles. The Guest Index field in the target must be a value between 0 and NUM_GEILEN.
4	v1.0.0	03/19/2025	Shota Minato	Initial public release.
5	v1.0.1	03/31/2025	Shota Minato	Added a new sections: the smoke test environment, the coverage tests, the versions of source codes and tools, and the verification result summary. Fixed the section links. Updated the random test section.
6	v1.0.2	06/03/2025	Shota Minato	Applied the new template.